

Software Engineering

Concise Study Notes: Chapters 1–9

ENCT 352 Year III, Part II

Scope. These notes follow the full syllabus for Software Engineering (ENCT 352, 45 hours, 60 theory marks): Chapter 1 Introduction, Chapter 2 The Software Process, Chapter 3 Software Requirements Engineering, Chapter 4 Architectural Design, Chapter 5 System Modeling (DFD, use case, activity and class diagrams), Chapter 6 Coding and Testing, Chapter 7 Software Quality, Assurance and Maintenance, Chapter 8 Software Configuration Management, and Chapter 9 Recent Trends. Diagrams (DFDs, UML use-case/activity/class diagrams, process models, architecture styles) are drawn out in full with worked examples, and logic-based testing techniques (equivalence partitioning, boundary value analysis) are worked step by step. Forty MCQs with an answer key are at the end.

Contents

	Chapter	Topics	Page
1	Introduction	Software and software engineering; characteristics; application domains; legacy software; software crisis; software myths; essence of practice	1
2	The Software Process	Process framework and umbrella activities; waterfall, incremental, evolutionary models; Agile manifesto, Scrum, XP, Lean; model selection	7
3	Software Requirements Engineering	Requirements process; SRS; functional/non-functional requirements; use cases and scenarios; agile requirements (user stories, backlog, story mapping)	19
4	Architectural Design	Design principles; taxonomy of architectural styles; modular design, cohesion and coupling	27
5	System Modeling	Need for modeling and abstraction; DFDs; scenario-based analysis and use-case diagrams; activity diagrams; class-based modeling	33
6	Coding and Testing	Coding standards; code review (walkthrough, inspection, Cleanroom); testing levels; black-box and white-box testing; agile testing (TDD)	41
7	Software Quality, Assurance, Maintenance	Quality concepts and attributes; reviews and SQA; ISO standards and CMMI; maintenance types and effort	49
8	Software Configuration Management	SCM concepts; version control and CI; change management; branching strategies	55
9	Recent Trends	DevOps and CI/CD; cloud-native and microservices; infrastructure as code; AI-assisted development; low-code/no-code; green software engineering	58
	Knowledge check	40 MCQs with answer key at the end	61

Chapter 1 Introduction

1.1 Software and Software Engineering

Software

Software is (1) instructions (computer programs) that, when executed, provide the desired features, function, and performance; (2) data structures that enable the programs to adequately manipulate information; and (3) descriptive information, in both hard-copy and virtual forms, that describes the operation and use of the programs.

Software Engineering (IEEE)

Software engineering is the application of a systematic, disciplined, quantifiable approach to the development, operation, and maintenance of software – that is, the application of engineering to software.

Modern software engineering emerged to address recurring problems of the pre-engineering era: late delivery, cost overruns, unreliable software, and difficulty maintaining existing systems. Treating software development as an engineering discipline – with defined processes, methods, and tools, rather than ad-hoc coding – is the direct response to these problems (see §1.5, Software Crisis).

Software engineering vs. computer science. Computer science focuses on the theories, algorithms, and fundamentals of how computers and programming languages work, often independent of business constraints. Software engineering focuses on the practicalities of developing, delivering, and maintaining useful software systems *within* constraints such as time, budget, and customer needs.

Software engineering as a layered technology

Software engineering rests on four layers. **Quality focus** is the bedrock: an organisational commitment to continuous quality improvement (e.g. TQM, Six Sigma) – reducing defects, improving customer satisfaction, and ensuring reliability and maintainability. **Process** defines the framework for management control, milestones, and change management, answering *what* activities are done, in what order, by whom, and how progress is tracked. **Methods** supply the technical how-to's – communication, requirements analysis, design modelling, construction, testing, and support (e.g. UML, OOAD, database modelling, design patterns). **Tools** give automated or semi-automated support for process and methods (CASE tools) – e.g. Git for source control, Jira for task management, Jenkins for deployment, Selenium for testing.

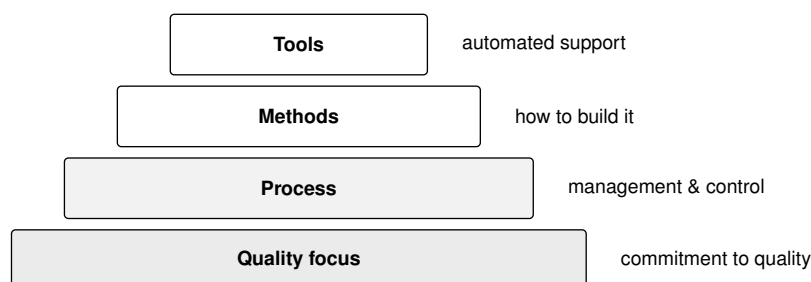


Figure 1: Software engineering layers: quality focus underlies process, which underlies methods, which underlies tools.

1.2 Nature and Characteristics of Software

Software is a **logical** element, not a physical one, which makes it fundamentally different from hardware.

- **Software is developed or engineered; it is not manufactured.** Although similarities exist between software development and hardware manufacturing, the two activities are fundamentally different: software quality is achieved through good design, not through production control.
- **Software doesn't "wear out".** Hardware follows a **bathtub curve**: a high early failure rate (infant mortality), a steady low-failure plateau, then rising failures as components physically wear out. Software has no physical wear; its idealised failure curve starts high (undiscovered defects) and then drops and flattens as defects are found and corrected. In practice, however, software *deteriorates*: each change introduces new errors, causing failure-rate spikes, so the actual baseline failure rate rises over time.
- **Software has no spare parts.** Every software failure indicates a design error; unlike hardware, where a worn component is swapped for a spare, software maintenance means fixing the design flaw itself – inherently more complex than replacement.

- **Most software is custom-built.** It is not assembled from standard components, although the industry is steadily moving toward component-based construction and reuse.

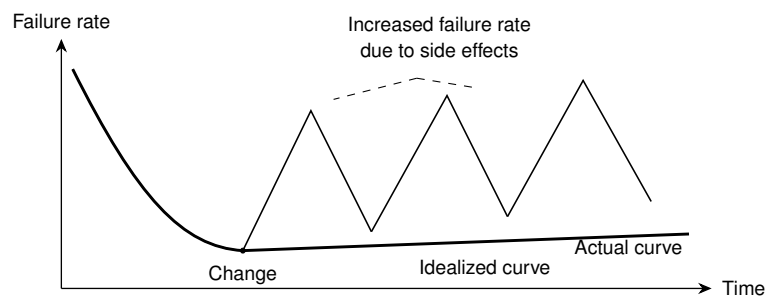


Figure 2: Failure curves for software: the idealized curve flattens after early defects are fixed; the actual curve spikes at each maintenance change, trending upward over time – unlike hardware’s bathtub curve.

Characteristics of good software. **Functionality** – delivers the required features. **Reliability** – performs without failure under stated conditions. **Usability** – easy for the intended users to learn and operate. **Efficiency** – makes optimal use of system resources. **Maintainability** – easy to modify, correct, and improve. **Portability** – can operate across different environments.

1.3 Software Application Domains

Computer software is commonly grouped into seven broad categories:

Domain	Description	Example
System software	Programs that service other programs; process complex, often indeterminate, information structures.	Compilers, OS components, drivers
Application software	Standalone programs solving specific business needs; process business/technical data for decisions.	Payroll, inventory
Engineering/scientific software	“Number-crunching” programs for scientific computation.	CAD, weather forecasting
Embedded software	Resides within a product to control its features and functions.	Automobile fuel control, microwave keypad
Product-line software	Reusable components built for many customers.	Word processors, spreadsheets
Web/mobile applications	Network-centric software spanning browser, cloud, and mobile platforms.	E-commerce sites, mobile apps
Artificial intelligence software	Simulates human intelligence to solve problems not amenable to straightforward algorithms, using heuristics.	Robotics, machine learning

1.4 Legacy Software

Legacy software

Legacy software refers to older programs, often developed decades ago, that have been continually modified to meet changing business requirements and computing platforms.

Legacy systems often support core business functions and are indispensable, yet are costly to maintain and risky to evolve: they may have inextensible designs, convoluted code, poor or absent documentation, no archived test cases, and a poorly managed change history. A legacy system that still

meets users' needs and runs reliably does not need to be fixed; but when evolution *is* needed, it must be **reengineered** for future viability – typically because it must be:

- adapted to new computing environments or technology;
- enhanced to meet new business requirements;
- extended to work with modern systems or databases; or
- re-architected to remain viable in an evolving environment.

Lehman's Laws of Software Evolution

Law of Continuing Change: a system must be continually adapted, or it becomes progressively less useful. **Law of Increasing Complexity:** as a system evolves, its structure degrades and its complexity increases, unless active work is done to maintain or simplify it.

1.5 Software Crisis

Software crisis

The **software crisis** is a term coined in the late 1960s (at the 1968 NATO Software Engineering Conference) to describe the chronic problems in software development – specifically, the inability to produce useful, reliable software on time and within budget as systems grew in complexity.

Causes of the software crisis. **Increasing complexity** of requirements against primitive tools and techniques. **Rising demand** for new software outpacing the industry's ability to produce it. **Inadequate techniques** – development that was code-first, informal, and dependent on a few individuals' knowledge. **Poor management** – unrealistic deadlines and inadequate quality assurance. **Maintenance challenges** – maintenance cost often exceeding initial development cost. **Shortage of practitioners** trained in software development. **Poor communication** between stakeholders, developers, and users, producing software that missed real business requirements.

These causes manifested as projects running significantly over budget and past deadlines; delivered software that failed to meet customer requirements; software that was error-prone, unreliable, and poorly documented; and large projects becoming unmanageable or being cancelled outright. Four well-documented historical cases illustrate the crisis:

Case	Year	What went wrong
IBM OS/360	1963–65	Massive delays and cost overruns in building a unified operating system.
Therac-25	1985–87	Software-based safety failures in a radiation-therapy machine caused patient fatalities.
Denver Airport baggage system	1995	Automated baggage-handling software glitches delayed the airport's opening by 16 months.
Ariane 5 rocket	1996	Crashed 37 seconds after launch due to an integer overflow in the control software.

Proposed solutions centre on adopting systematic software engineering practices (process models, methods, and tools); better requirements gathering and management; formal reviews, inspections, and testing strategies; standards and quality-assurance frameworks; and training and education in software engineering principles.

1.6 Software Myths

Software myths are widely held but false beliefs about software and its development that lead to mismanagement, unrealistic expectations, and project failures. They fall into three categories: management myths, customer myths, and practitioner (developer) myths.

Management myths.

- *Myth:* “We already have a book full of standards and procedures for building software – won’t that provide my people with everything they need to know?”
Reality: standards may exist but are often incomplete, outdated, or simply not used; a book does not guarantee that practice is adequate or that practitioners follow it.
- *Myth:* “If we get behind schedule, we can add more programmers and catch up.”
Reality: adding people to a late software project makes it later – **Brooks’ Law**. New staff need time to learn the system, and communication overhead grows quadratically with team size.
- *Myth:* “If I decide to outsource the software project, I can just relax and let the vendor build it.”
Reality: an organisation that cannot manage and control software development internally will invariably struggle when outsourcing; oversight and involvement remain essential.

Customer myths.

- *Myth:* “A general statement of objectives is sufficient to begin writing programs – we can fill in the details later.”
Reality: ambiguous requirements are the primary cause of failed projects; a formal, detailed requirements specification is essential to minimise rework and miscommunication.
- *Myth:* “Software requirements continually change, but changes can be easily accommodated because software is flexible.”
Reality: software is malleable, but the cost of change rises sharply as development progresses – a change made during design can be 1.5–6× more costly than the same change during requirements, and during testing it can be 60–100× more costly.

Practitioner (developer) myths.

- *Myth:* “Once we write the program and get it to work, our job is done.”
Reality: 60–80% of total effort is expended *after* first delivery, during maintenance (bug fixing, enhancement, adaptation).
- *Myth:* “Until I get the program running, I have no way of assessing its quality.”
Reality: quality can be assessed from inception, through formal technical reviews, design analysis, and inspections – long before a single line of code is compiled.
- *Myth:* “The only deliverable for a successful project is the working program.”
Reality: documentation, models, test plans, and design specifications are equally critical deliverables for long-term maintainability and project success.

1.7 Software Engineering Practice

The essence of practice (Polya’s problem-solving steps)

Based on George Polya’s problem-solving approach (*How to Solve It*, 1945), software engineering practice follows four essential steps:

1. **Understand the problem** (communication and analysis) – who are the stakeholders, what are the unknowns, what data/functions/features are required, and can the problem be decomposed and represented graphically?
2. **Plan a solution** (modelling and software design) – have similar problems been solved before, can sub-problems be defined with apparent solutions, and can a design model be built that leads to effective implementation?
3. **Carry out the plan** (code generation) – does the solution conform to the design plan, is the code traceable to the design model, and has each component been reviewed?
4. **Examine the result for accuracy** (testing and quality assurance) – has a reasonable testing strategy been used, do the results conform to requirements, and has the software been validated against all stakeholder requirements?

Hooker's Seven Principles.

1. **The Reason It All Exists** – a system exists to provide value to its users; if a feature adds no real value, don't build it.
2. **KISS (Keep It Simple, Stupid!)** – design should be as simple as possible, but no simpler; simpler designs are easier to understand and maintain.
3. **Maintain the Vision** – a clear architectural vision, held by an empowered architect, prevents a patchwork of incompatible designs.
4. **What You Produce, Others Will Consume** – design, code, and document knowing someone else must understand and maintain the work.
5. **Be Open to the Future** – never design into a corner; solve the general problem, not just the specific one.
6. **Plan Ahead for Reuse** – reduces cost and increases the value of both the reusable components and the systems that use them.
7. **Think!** – clear, complete thought before action produces better results, and helps recognise when more research is needed.

Chapter 2 The Software Process

2.1 Process Framework and Umbrella Activities

Software process

A **software process** is a collection of activities, actions, and tasks performed when a work product is to be created. It is not rigid but adaptable to the problem, the project, the team, and the organizational culture.

Framework activities

A generic **process framework** defines five framework activities that apply to all software projects, regardless of size or complexity:

1. **Communication** – collaborating with stakeholders to understand their objectives and gather requirements that define software features and functions.
2. **Planning** – creating a software project plan that describes technical tasks, risks, resources, work products and schedule (the project “road map”).
3. **Modeling** – creating models (sketches, UML diagrams, architectural representations) to understand requirements and the design that will achieve them.
4. **Construction** – code generation (manual or automated) combined with testing to uncover errors in the code.
5. **Deployment** – delivering the software (complete or as an increment) to the customer, who evaluates it and provides feedback.

These activities are applied *iteratively*: each iteration produces a software increment that gives stakeholders a subset of the overall features, until the software is complete.

Process flow

Process flow describes how the framework activities are organized with respect to sequence and time; the choice depends on the project’s requirements stability and the team’s familiarity with the domain.

- **Linear process flow** – executes all five activities once, in strict sequence (the basis of the waterfall model, Section 2.2.1).
- **Iterative process flow** – repeats one or more activities (e.g. short loops between Planning/Communication and Modeling) before proceeding, plus one large loop back to Communication after Deployment.
- **Evolutionary process flow** – executes the activities in a circular manner; each circuit around the loop releases a more complete increment (the basis of Prototyping and Spiral, Section 2.2.3).
- **Parallel process flow** – executes one or more activities concurrently with others (e.g. modeling one subsystem while constructing another).

Umbrella activities

Umbrella activities are applied throughout the software process to manage and control progress, quality, change and risk, regardless of which process flow is chosen:

1. **Software project tracking and control** – assesses progress against the project plan and triggers corrective action.
2. **Risk management** – assesses risks that may affect project outcome or product quality.
3. **Software quality assurance (SQA)** – defines and conducts activities that ensure software quality.
4. **Technical reviews** – uncover and remove errors before they propagate to the next activity.

5. **Measurement** – collects process, project and product measures to help deliver software that meets stakeholder needs.
6. **Software configuration management (SCM)** – manages the effects of change throughout the process.
7. **Reusability management** – defines criteria for work-product reuse and sets up mechanisms for reusable components.
8. **Work product preparation and production** – creates models, documents, logs, forms and lists.

2.2 Traditional (Plan-Driven) Process Models

Plan-driven (prescriptive) process models

Traditional (prescriptive) process models define a predefined set of process elements and a predictable process workflow. They prescribe framework activities, engineering actions, tasks, work products, quality assurance and change-control mechanisms, and strive for structure and order in software development.

Waterfall Model and Its Extensions

Waterfall model

The **waterfall model** (also called the **linear sequential model**), proposed by Winston Royce (1970), is the oldest and most classical Software Development Life Cycle (SDLC) paradigm. It suggests a systematic, sequential flow through Communication, Planning, Modeling, Construction and Deployment: *each phase must be completed before the next begins*, with little opportunity to revisit earlier phases.

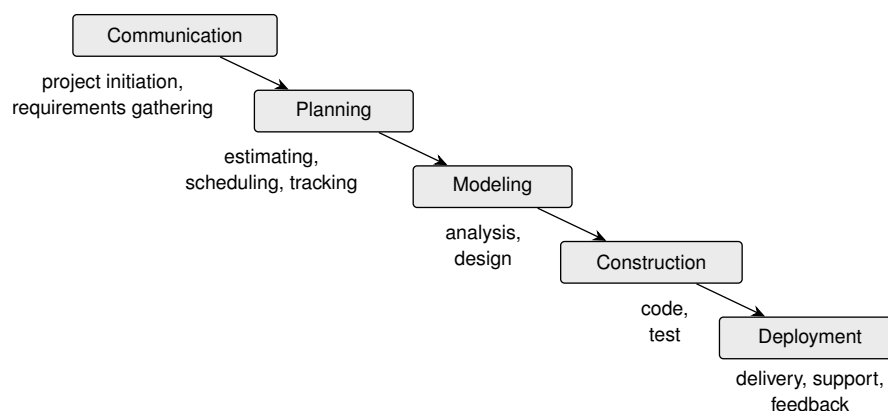


Figure 3: The waterfall model: a strict sequential cascade through the five framework activities.

Advantages	Disadvantages
• Simple to understand and easy to plan. • Works well for small, well-understood projects with stable requirements. • Analysis and testing are straightforward. • Clear milestones and deliverables at each phase.	• Real projects rarely follow a strictly sequential flow. • Hard for the customer to state all requirements explicitly upfront. • No working software is available until late in the project. • Major errors may go undetected until the working program is reviewed. • Cost of change increases dramatically if a defect is found late. • Testing occurs late, and customer approval is only at the end.

Example: Why is the waterfall model unsuitable when key functionality must be delivered quickly?

Because all phases must complete sequentially, no working software is delivered until Construction is finished. If important features are needed fast, waterfall offers no mechanism for partial or incremental release – the customer must wait for the entire system before seeing any working functionality.

Extensions of the waterfall model

- **V-Model** – associates a dedicated testing phase with each development phase (e.g. unit testing validates module design, system testing validates system design), making verification explicit rather than a single late Construction step.
- **Waterfall with feedback** – allows limited iteration between *adjacent* phases, partially relaxing the rigidity of the pure waterfall.

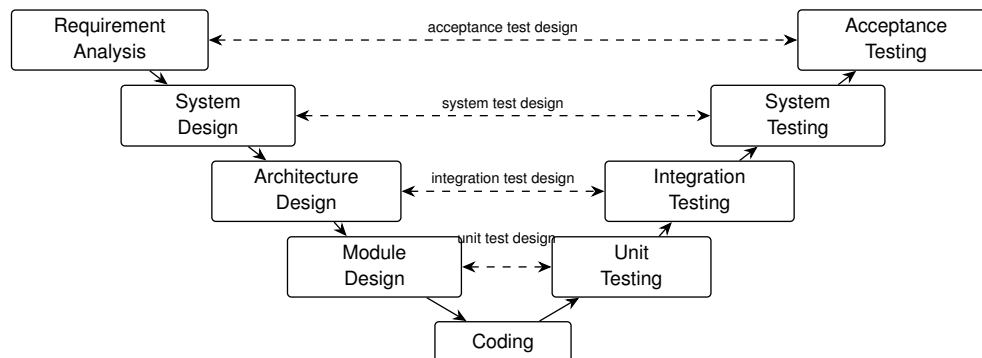


Figure 4: V-Model: each development phase (left arm) is paired with a corresponding testing phase (right arm) at the same level of detail.

Incremental Process Model

Incremental model

The **incremental model** combines elements of the linear and iterative process flows. It delivers software in a series of **increments**, each providing a portion of the overall functionality. The first increment is usually a core product addressing basic requirements; subsequent increments add supplementary features.

Requirements are gathered and prioritized up front, then the system is divided into increments; each increment goes through its own mini-waterfall cycle and delivers a working, usable product, and customer feedback from one increment informs the planning of the next.

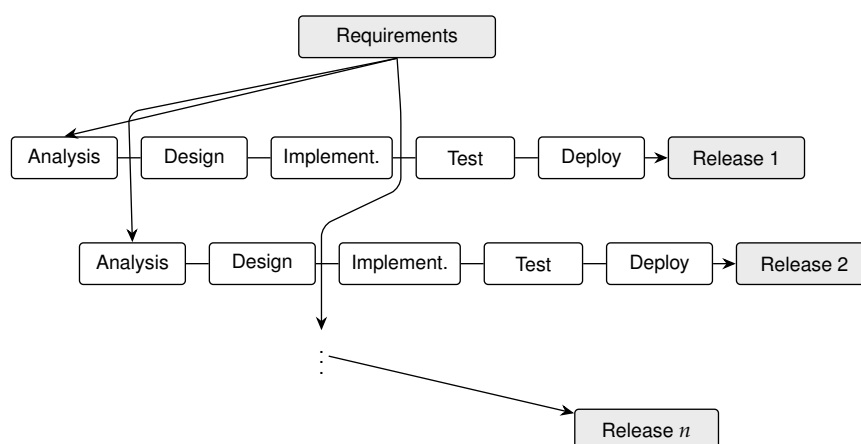


Figure 5: Incremental model: requirements are partitioned across increments; each increment runs its own mini-cycle (analysis, design, implementation, test, deploy) before release. (This figure's stage names follow the source diagram's own convention; they correspond to Modeling, Modeling, Construction, Construction and Deployment in the five-framework-activity vocabulary used elsewhere.)

Example: E-commerce site, incremental approach. A team builds an online store one increment at a time, each fully functional, deployed, tested and reviewed by stakeholders before the next begins:

- **Increment 1** – core functionality: user registration, product-catalogue browsing, basic search.
- **Increment 2** – shopping cart, checkout process, online payment integration.
- **Increment 3** – user reviews, recommendation engine, order tracking.
- **Increment 4** – admin dashboard, analytics, inventory management, promotional tools.

Advantages	Disadvantages
• Delivers working software early, providing value sooner. • Customer feedback after each increment reduces the risk of product rejection. • Smaller modules are easier to test and debug. • Flexible – easier to accommodate requirement changes between increments. • Higher-risk features can be addressed in early increments. • Lower initial delivery cost.	• Requires a clear understanding of the complete system to partition it properly. • Requires careful planning and architectural foresight. • Total cost can exceed a single-pass model, since each increment has its own full cycle. • Integration complexity grows as more increments are added.

Evolutionary Process Models (Prototyping, Spiral)

Prototyping model

Prototyping model

The **prototyping model** is used when requirements are fuzzy, incomplete or poorly understood. A **prototype** is a preliminary version of the software, built quickly, that helps stakeholders visualize and refine their requirements. All stakeholders should agree upfront that the prototype defines requirements – it is *not* the final product.

Its process runs: (1) **Communication** – meet stakeholders to define objectives and known requirements; (2) **Quick plan** – plan a rapid prototyping iteration; (3) **Modeling (quick design)** – focus on what end users will see (UI layout, output formats); (4) **Construction of prototype** – build a working prototype quickly, reusing fragments or rapid-application tools; (5) **Deployment and feedback** – deliver the prototype for evaluation; (6) **Iteration** – tune the prototype and repeat until requirements are sufficiently understood.

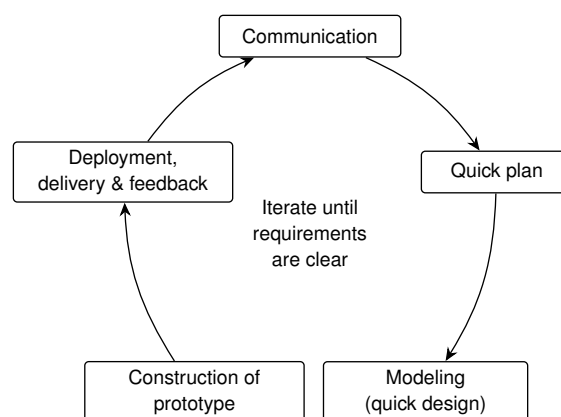


Figure 6: The prototyping paradigm: a quick prototype is built, evaluated and refined through successive iterations.

Prototyping is used (1) when the customer has general objectives but cannot specify detailed requirements, (2) when the developer is unsure about the form of user interaction, algorithm efficiency,

or system behaviour, and (3) for systems with heavy UI requirements where users need to “see and feel” before specifying details.

Example: Travel agency booking application. A travel agency needs a booking application but is unsure about the user-interface design. A prototype lets the agency interact with a preliminary UI, give feedback on layout and workflow, and progressively refine the design until it meets expectations – rather than the agency trying to specify the exact UI on paper before any code exists.

Advantages	Disadvantages
• Reduces impact of requirement changes, since requirements are refined iteratively. • Customer is involved early and often. • Reduces likelihood of product rejection. • Uncovers developer–stakeholder misunderstandings early. • Works well for small to medium projects.	• Stakeholders may mistake the prototype for the finished product and expect immediate delivery. • Developers may embed poor architecture or quick fixes made just to get the prototype working. • Work is lost if the prototype is thrown away. • Hard to plan and manage, since the number of iterations is unpredictable.

Spiral model

Spiral model

The **spiral model**, proposed by Barry Boehm (1988), is an evolutionary process model that couples the iterative nature of prototyping with the controlled, systematic discipline of the waterfall model. Its distinguishing feature is *explicit risk analysis at every iteration*.

Software is developed as a series of evolutionary releases, traversing a spiral path through five framework activities each loop: **Communication** (objectives and constraints); **Planning/risk analysis** (alternatives, cost and schedule estimates, risk identification and resolution, prototypes to reduce uncertainty); **Modeling** (analysis and design, progressively refined each pass); **Construction** (developing and verifying the software); **Deployment** (delivering the increment, assessing results, planning the next loop). Each circuit produces a progressively more complete version of the software – the first might yield only a specification or proof-of-concept prototype, later circuits yield increasingly refined, complete versions.

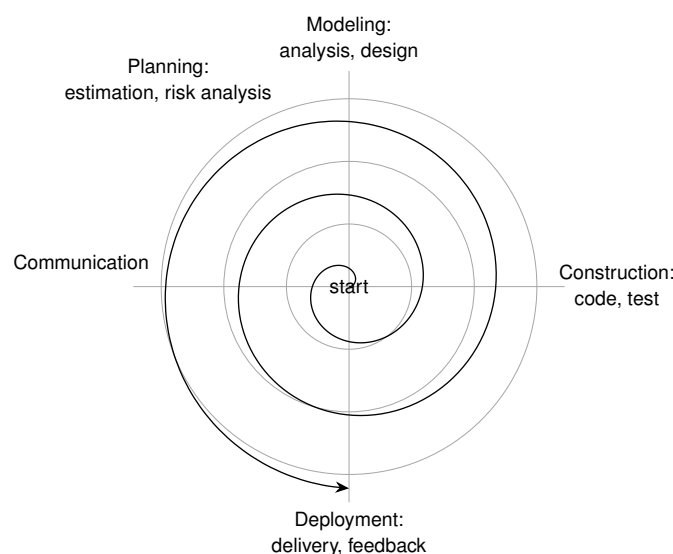


Figure 7: The spiral model: each loop outward through the activity quadrants produces a progressively more complete (and more costly) version of the system, with risk analysis performed on every loop.

The spiral is a *meta-model* that can accommodate other process models within it: the first circuit can use a prototyping approach to explore uncertain requirements and refine the UI; once requirements

stabilize, later circuits can follow a more waterfall-like sequential flow. Thus the spiral uses prototyping for risk reduction on uncertain portions, and waterfall-like discipline on well-understood portions – each loop’s approach is chosen by that iteration’s risk profile.

Advantages	Disadvantages
• Continuous customer involvement throughout the project. • Development risks are explicitly managed at every iteration. • Suitable for large, complex, high-risk projects. • Works well for extensible products that evolve over time. • Can be applied throughout the entire lifecycle, including maintenance.	• Risk-analysis failures can doom the project. • Hard to convince customers the evolutionary approach is controllable. • Requires considerable risk-assessment expertise. • Evolving scope can make the project hard to manage. • Not suitable for small, low-risk projects – overhead is disproportionate.

Example: Travel agency, UI uncertainty – would the spiral model be proper? The same travel-agency booking application (uncertain UI) can also be framed for the spiral model: the first loop builds a UI prototype to resolve the UI-design risk, and stakeholder feedback refines the UI requirements; subsequent loops then address core booking logic, payment integration and database design, each with progressively less risk. Because the spiral model is risk-driven, the biggest uncertainty (the UI) is resolved first, before larger amounts of the budget are committed – this is the key difference from plain prototyping, which iterates on the UI but performs no formal risk analysis on the rest of the system.

2.3 Agile and Adaptive Process Models

Where plan-driven and evolutionary models emphasize predictability and risk control, **agile** models emphasize speed, flexibility and continuous customer feedback.

Agile development

Agile is an umbrella term for adaptive, iterative process models that emerged from the **Agile Manifesto** (2001), written by 17 software practitioners reacting against the rigidity of plan-driven models. Agile processes deliver working software in short cycles, welcome changing requirements, and rely on close, continuous collaboration with the customer.

Agile Manifesto and the 12 Principles

The manifesto states four value statements. While the items on the right have value, agile teams value the items on the left more:

1. Individuals and interactions *over* processes and tools.
2. Working software *over* comprehensive documentation.
3. Customer collaboration *over* contract negotiation.
4. Responding to change *over* following a plan.

These values are elaborated by twelve supporting principles:

1. Satisfy the customer through early, continuous delivery of valuable software.
2. Welcome changing requirements, even late in development – agile harnesses change for the customer’s competitive advantage.
3. Deliver working software frequently (weeks rather than months), preferring the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals; give them the environment and support they need, and trust them.
6. Face-to-face conversation is the most efficient way to convey information within a team.
7. Working software is the primary measure of progress.
8. Sustainable development – sponsors, developers and users should maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – maximizing the amount of work *not* done – is essential.
11. The best architectures, requirements and designs emerge from self-organizing teams.
12. At regular intervals the team reflects on how to become more effective, then tunes its behaviour accordingly.

Agile versus Plan-Driven Development

Plan-driven models follow a predefined phase sequence with detailed upfront planning, comprehensive documentation and milestone tracking; requirements are gathered completely before design, and change goes through formal change control. **Agile** models instead emphasize adaptability and continuous feedback: documentation stays lightweight, requirements emerge through ongoing customer collaboration, and change is embraced within short iterations.

Dimension	Plan-driven	Agile
Planning	Predefined sequence, detailed upfront	Adaptive, iterative
Documentation	Comprehensive, formal	Lightweight; working software is primary
Requirements	Gathered completely upfront	Emerge and evolve through continuous collaboration
Change management	Formal change-control process	Embraced via short iterations
Best fit	Stable requirements; safety/mission-critical; large, distributed team; strong process culture	Volatile requirements; small, co-located, experienced team; rapid delivery critical

A key practical difference is the **cost of change**: in plan-driven development this cost rises exponentially as the project progresses (a change caught during testing can be 60–100× costlier than one caught during requirements), whereas agile practices – continuous testing, incremental delivery, refactoring, close customer contact – aim to flatten this curve. In practice many projects combine both: agile flexibility for day-to-day work, with enough planning discipline for coordination and risk management.

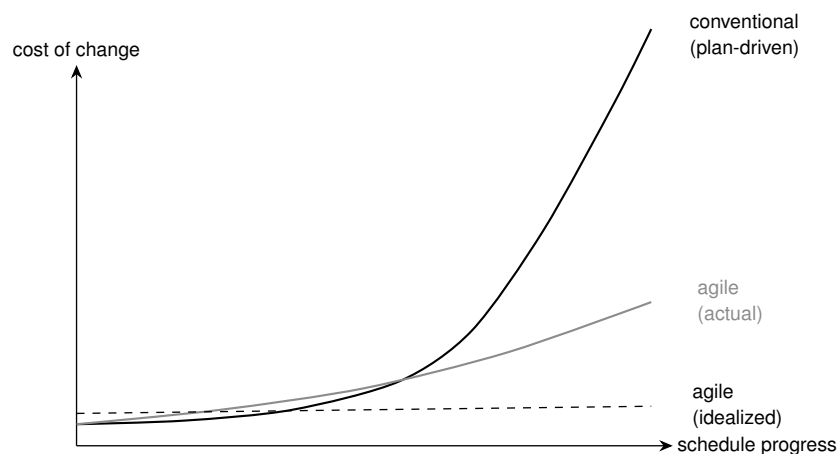


Figure 8: Cost of change versus schedule progress: plan-driven cost rises steeply late in the project, agile’s actual cost rises far more gently, and the theoretical idealized agile cost stays flat.

Scrum Framework

Scrum

Scrum, conceived by Jeff Sutherland and formalized by Ken Schwaber and Mike Beedle, is the most widely used agile framework. Work proceeds in fixed-length, time-boxed iterations called **sprints** (typically 2–4 weeks), each producing a usable increment of the product.

Roles.

- **Product Owner** – represents the customer and stakeholders; owns and prioritizes the Product Backlog by business value, sets sprint goals, and alone decides whether to accept or reject an increment.
- **Scrum Master** – a servant-leader, not a project manager; coaches the team on Scrum practice, facilitates the Daily Scrum, and removes impediments blocking progress.
- **Development Team** – a small (3–9 people), self-organizing, cross-functional team that designs, builds and tests the increment, deciding among itself how selected backlog items are implemented.

Artifacts.

- **Product Backlog** – a prioritized, continuously evolving list of all features, requirements, enhancements and fixes; never “complete” while the product exists.
- **Sprint Backlog** – the subset of Product Backlog items selected for the current sprint plus the team’s plan for delivering them; closed to new items once the sprint starts.
- **Increment** – the sum of all completed backlog items from the current and prior sprints, in a usable, potentially releasable state meeting the team’s Definition of Done.

Ceremonies.

- **Sprint Planning** – start of the sprint; the Product Owner presents the goal, and the team selects and estimates items to form the Sprint Backlog.
- **Daily Scrum** – a 15-minute daily stand-up where each member states what was done, what will be done, and what obstacles exist.
- **Sprint Review** – end of the sprint; the team demonstrates the increment to the Product Owner and stakeholders, who give feedback and accept or reject it.
- **Sprint Retrospective** – held after the review; the team reflects on what went well and what to improve, committing to specific process changes.

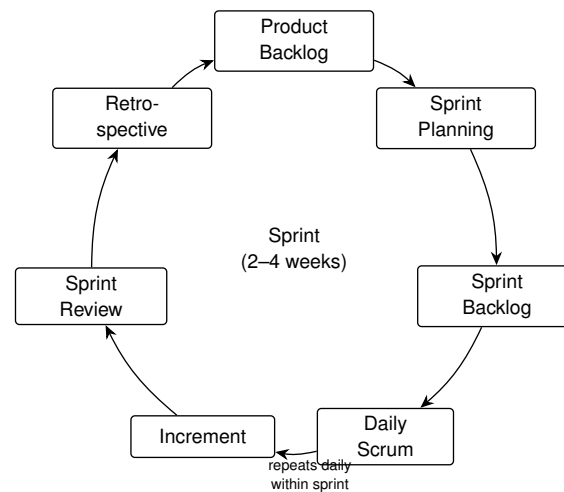


Figure 9: Scrum sprint cycle: the Product Backlog feeds Sprint Planning, producing the Sprint Backlog worked through daily Scrums; the sprint ends with an Increment, a Sprint Review and a Retrospective before the cycle repeats.

Extreme Programming (XP) Practices

Extreme Programming (XP)

Extreme Programming (XP), created by Kent Beck, pushes proven development practices to their most effective (“extreme”) levels. It is organized around four framework activities – planning, design, coding and testing – guided by five core values: **Communication, Simplicity, Feedback, Courage and Respect**.

XP prescribes thirteen concrete practices:

1. **User Stories** – requirements as short customer-written narratives, each given a business value by the customer and a cost estimate by the team.
2. **Planning Game** – customers and developers jointly decide which stories enter the next release and in what order.
3. **Small Releases** – software is released in small, frequent increments so customers evaluate working software early.
4. **Simple Design** – keep it simple (KIS); no functionality is added on speculation about future needs.
5. **Pair Programming** – two developers share one workstation: a driver writes code while a navigator reviews it in real time.
6. **Test-Driven Development (TDD)** – write a failing test, write just enough code to pass it, then refactor (red-green-refactor).
7. **Refactoring** – continuously improve the code’s internal structure without changing its external behaviour.
8. **Continuous Integration** – integrate into the shared repository multiple times a day, with automated builds/tests catching errors early.
9. **Collective Code Ownership** – any developer may change any part of the codebase, avoiding knowledge silos.
10. **Coding Standards** – the team follows agreed conventions so the codebase reads as if written by one person.
11. **Sustainable Pace** – a 40-hour week; overtime is discouraged because a rested team produces better work.
12. **On-Site Customer** – a customer representative sits with the team to answer questions and set priorities.
13. **Project Velocity** – stories completed per iteration, used to estimate delivery dates for future releases.

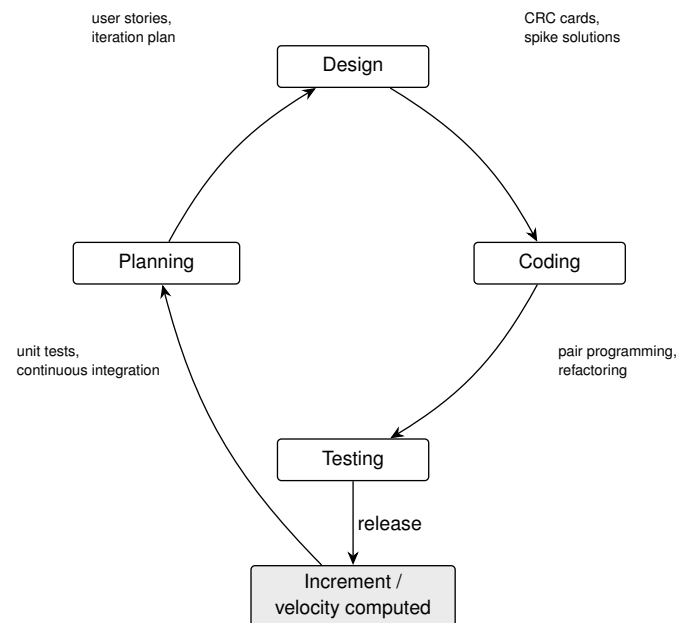


Figure 10: The XP process: planning, design, coding (with pair programming and continuous refactoring) and testing repeat every iteration; each cycle ends with a release and an updated velocity measurement feeding the next planning stage.

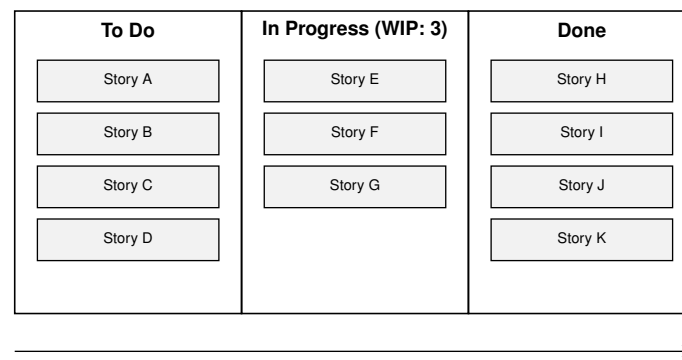
Lean Software Development

Lean software development

Lean software development, articulated by Mary and Tom Poppendieck, adapts the principles of lean manufacturing (the Toyota Production System) to software engineering, focusing on delivering maximum customer value with minimum waste.

1. **Eliminate waste** – remove anything that adds no customer value: partially done work, unnecessary features, task switching, waiting, handoffs, extra meetings, defects.
2. **Amplify learning** – treat development as a discovery process; use short iterations and frequent feedback to build knowledge rather than rely on rigid upfront plans.
3. **Decide as late as possible** – delay irreversible decisions until maximum information is available, keeping options open.
4. **Deliver as fast as possible** – shorten the time between identifying a need and delivering working software, for faster feedback and lower risk.
5. **Empower the team** – trust the people closest to the work with the authority, tools and environment to solve problems; avoid micro-management.
6. **Build integrity in** – ensure conceptual integrity (the system works as a cohesive whole) and perceived integrity (it meets expectations), through automated testing, continuous integration and refactoring.
7. **Optimize the whole** – improve the entire value stream from concept to delivery rather than individual parts, avoiding local sub-optimization.

Lean principles complement other agile methods; **Kanban**, a lean-originated method, visualizes work on a board, limits work in progress (WIP), manages flow explicitly, and creates feedback loops for continuous improvement.



work flows left to right; the board limits how many items may be "In Progress" at once

Figure 11: A simple Kanban board: cards move from To Do through In Progress (bounded by a WIP limit) to Done, making flow and process policy explicit.

2.4 Model Selection Considerations

Software process model

A **software process model** is a simplified description of a software process presented from a particular perspective; it defines the sequence of activities, their interdependencies, and how they transform inputs into outputs. No single model fits every project – the right choice depends on the project's size, complexity, requirements stability, risk, team, customer involvement and time constraints.

Seven factors guide the choice of process model:

1. **Requirements clarity and stability** – well-defined, stable requirements favour plan-driven models (e.g. waterfall); volatile or unclear requirements favour agile or evolutionary models.
2. **Project size and complexity** – large, complex, high-risk projects favour the spiral model; small to medium projects favour agile methods such as Scrum or XP.
3. **Risk level** – high-risk projects benefit from the spiral model's explicit risk analysis; low-risk projects can use simpler models.
4. **Customer involvement** – a customer available for continuous participation favours agile; limited customer interaction favours plan-driven models with formal documentation.
5. **Time-to-market pressure** – when rapid delivery is critical, incremental or agile models are preferred, since they deliver working software early.
6. **Team expertise** – agile and spiral models need experienced, self-organizing teams; plan-driven models can accommodate less experienced teams given strong management.
7. **Regulatory and compliance needs** – safety-critical or regulated domains (avionics, medical devices) may require the documentation rigour of plan-driven models.

Example: For a project with significant uncertainties and risk of loss, the spiral model is most appropriate: its risk-driven nature identifies and resolves uncertainties early through prototyping and formal risk analysis before major resources are committed, and the project can be terminated at any loop if risk proves too high – limiting potential losses. A **feasibility study**, conducted before any project is accepted, similarly guards against this risk by assessing technical, economic and operational viability up front, examining resource availability, cost–benefit and timeline constraints so the organization avoids committing to projects unlikely to succeed.

Spiral Model	Incremental Model
Performs formal risk analysis at every iteration	Manages risk informally, by prioritizing increments
Customer is involved continuously, especially in planning and risk assessment	Customer provides periodic feedback after each increment
Best suited for large, complex, high-risk projects	Best suited for medium projects with generally known requirements
Follows an evolutionary flow – each loop revisits all phases with risk analysis	Follows an iterative flow – each increment runs its own mini-lifecycle
Changes are incorporated after risk assessment in each loop	Changes are easier to accommodate between increments
Requires moderate documentation, including risk reports	Documentation varies and can be lightweight
Requires risk-assessment expertise and is expensive for small projects	Requires complete system understanding for proper partitioning

Chapter 3 Software Requirements Engineering

3.1 Requirement Engineering Process

Requirements engineering

Requirements engineering (RE) is the broad spectrum of tasks and techniques that lead to an understanding of requirements. It begins during the communication activity and continues into modeling, establishing the solid base on which design and construction rest. Without it, the resulting software has a high probability of not meeting customer needs.

RE is organised into seven tasks, carried out roughly in sequence but frequently revisited as understanding improves – especially the loop between elicitation, elaboration and negotiation, which repeats until stakeholders agree on a stable set of requirements.

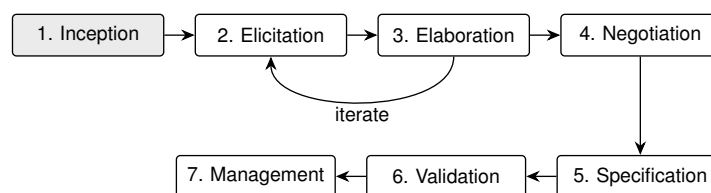


Figure 12: The seven tasks of requirements engineering; elicitation, elaboration and negotiation loop until stakeholders agree.

- **Inception** – establish a basic understanding of the problem, the stakeholders who want a solution, and the nature of the desired solution; project scope is outlined and initial communication established.
- **Elicitation** – actively gather requirements from stakeholders: what the system must accomplish, how it fits business needs, how it will be used daily; understand and prioritise business goals (functional and non-functional).
- **Elaboration** – develop a refined requirements model identifying software function, behaviour and information; user scenarios are parsed to extract analysis classes, attributes and services, described only in enough detail to support design.
- **Negotiation** – reconcile conflicting requirements among stakeholders so requirements are ranked, conflicts discussed and a “win-win” outcome reached; requirements are eliminated, combined or modified by cost, risk and priority.
- **Specification** – document the agreed requirements, as a written document, graphical models, a formal mathematical model, usage scenarios, a prototype, or a combination; large systems use a formal SRS, smaller products may use user stories or use cases.
- **Validation** – assess the quality of the requirements work products: unambiguous, consistent, with omissions and errors corrected, chiefly through a technical review with engineers, customers, users and other stakeholders.
- **Management** – identify, control and track requirements and their changes throughout the project lifecycle, since requirements change constantly and must be handled systematically (closely tied to configuration management, Chapter 8).

Understanding customer requirements is difficult in practice because customers often have only a vague idea of what is required, different stakeholders hold conflicting opinions and unspoken assumptions, requirements are stated in ambiguous or untestable terms (“the system should be user-friendly”), customers may not know exactly what they want until they see working software, and requirements keep changing throughout the project.

Requirements gathering techniques. **Interviews** (one-on-one/group, structured or unstructured); **facilitated meetings/JAD sessions** (a facilitator-led workshop where engineers and stakeholders jointly build and refine lists of objects, services, constraints and performance criteria); **prototyping** (a preliminary build so stakeholders can see and refine fuzzy requirements or UI concerns); **observation/ethnography** (watching users do their actual work to surface implicit requirements); **questionnaires/surveys** (quantitative preferences from many stakeholders at once); **document analysis** (studying manuals, forms, reports, regulations and legacy specifications).

A **software requirements document (SRD)**, commonly the **Software Requirements Specification (SRS)**, is a formal written understanding of the problem that all parties agree upon. It describes the required informational, functional and behavioural domains of the system, serves as the contractual basis between customer and developer, and is the foundation for design, testing and maintenance.

3.2 Software Requirements Specification (SRS)

The SRS matters because it gives a clear, shared understanding of what the software must do (removing ambiguity between customers, developers and testers), serves as a contractual agreement, underpins project planning, cost estimation and scheduling, provides traceability into design/implementation/testing, supports future maintenance, and reduces the risk of building the wrong product.

SRS structure (IEEE 830 / ISO/IEC/IEEE 29148)

1. **Introduction** – purpose of the SRS, scope of the software, definitions/acronyms, references, document overview.
2. **Overall description** – product perspective (context within a larger system), product functions (high-level summary), user characteristics, constraints (hardware, software, regulatory), assumptions and dependencies.
3. **Specific requirements** – functional requirements (detailed per function), external interface requirements (user, hardware, software, communication), performance requirements (response time, throughput), design constraints, and non-functional attributes (reliability, security, maintainability, portability).

Eight characteristics of a good SRS. **Correct** – accurately represents stakeholder needs. **Unambiguous** – each requirement has only one interpretation. **Complete** – all significant requirements present, no TBDs. **Consistent** – no requirement conflicts with another. **Verifiable (testable)** – every requirement can be checked by an objective process. **Modifiable** – structure allows easy change while keeping consistency. **Traceable** – each requirement can be traced to its source and to design/test elements. **Feasible** – achievable within budget, schedule and technology.

The SRS is written for, and used by, five distinct classes of readers: **customers/clients** (verify it captures their needs), **project managers** (planning, estimation, tracking), **developers/designers** (basis for design and implementation), **testers/QA** (derive test cases), and **maintenance engineers** (understand the system during later modification).

3.3 Functional and Non-Functional Requirements

Functional vs. non-functional requirements

Functional requirements (FR) describe *what* the system must do – the specific functions, features, services and behaviours it must provide, and its response to given inputs under given conditions. *Example: “The system shall allow users to search flights by origin, destination, and date.”*

Non-functional requirements (NFR) describe *how well* the system must perform – quality attributes, performance characteristics, security constraints and general constraints on the system, rather than

specific features.

Functional requirements	Non-functional requirements
Describe what the system does	Describe how well the system performs
Specify specific features and behaviours <i>Example: "User can place an order"</i>	Specify quality attributes and constraints <i>Example: "Order confirmation within 2 seconds"</i>
Tested by executing the function directly	Tested by measuring performance, stress, security, etc.
If absent, the system is incomplete – required features are missing	If absent, the system becomes unreliable, slow or insecure
Documented as use cases, user stories or feature lists	Documented as quality metrics, constraints or SLA targets

Both kinds of requirement belong in the SRS: FRs alone do not guarantee a usable system (it may do everything asked yet be too slow, insecure or hard to use), and NFRs without the required features are meaningless. Together they give a complete specification developers can build to, both functional and non-functional test cases, measurable acceptance criteria for the contract, and protection against the costly rework that missing NFRs cause.

User requirements are high-level, natural-language statements (possibly with diagrams) of what the system should provide, written for customers and managers – *Example: "The system should allow customers to book flights."* **System requirements** are detailed, precise descriptions of functions, services and constraints, written for developers and testers – *Example: "The system shall accept origin (IATA code), destination (IATA code), departure date (YYYY-MM-DD) and number of passengers (1–9) as search parameters and return matching flights sorted by price within 3 seconds."*

Seven categories of NFR. **Performance** (response time, throughput, capacity – *Example: "page loads within 2s under 1000 concurrent users"*); **reliability** (MTBF, availability – *Example: "99.9% uptime"*); **usability** (ease of use/learning – *Example: "a new user completes registration within 3 minutes unaided"*); **security** (authentication, authorisation, encryption – *Example: "passwords stored using bcrypt hashing"*); **maintainability** (ease of modification, modular design); **portability** (operates across platforms); **scalability** (handles growth in users, data or transactions).

Worked example: Online Food Ordering System

This system is used again in Section 3.4 for its use-case diagram, so the requirements below carry through as the same running example.

Functional requirements	Non-functional requirements
Allow the customer to browse the menu and select food items	Complete the payment transaction within 5 seconds
Allow the customer to specify quantity and customisations (e.g. extra cheese, no onions) per item	Transmit all payment data over encrypted connections
Allow the customer to add items to the cart	Store user passwords in hashed format
Show a cart summary with item-wise and total prices	Support a mobile-responsive design
Support multiple payment methods (card, digital wallet, cash on delivery)	Handle order processing without data loss during peak hours
Show an order confirmation with estimated delivery time after payment	Maintain 99.5% uptime during restaurant operating hours

Related worked examples covered by past exam questions on this topic, not repeated here in full: an **Airlines Reservation System** (search flights, book seat, PNR confirmation; NFRs on response time, PCI-DSS payment security, 10,000 concurrent users); a **Movie Theatre Ticket Booking System** (seat map, temporary seat hold, e-ticket with QR code); and a **Restaurant Information System** (menu display, dine-in/takeaway orders, table reservation, tablet-based waitstaff interface). Section 3.2's library book-issuing example (search, verify membership, check availability, record issue/due date) is a further instance of the same FR/NFR split.

Recap. RE = 7 tasks (Inception, Elicitation, Elaboration, Negotiation, Specification, Validation, Management), looping until stable. SRS = Introduction / Overall description / Specific requirements (IEEE 830), and must be correct, unambiguous, complete, consistent, verifiable, modifiable, traceable and feasible. Functional requirements say what the system does; non-functional requirements say how well it does it, across performance, reliability, usability, security, maintainability, portability and scalability.

3.4 Gathering Requirements Using Use Cases and Scenarios

Scenario and use case

A **scenario** is a specific narrative describing a single path of interaction between an actor and the system – a concrete story of how the system is used. A **use case** is a structured, generalised description of how an actor interacts with the system to achieve a specific goal; it includes the main success scenario plus alternate/exception flows, and may be expressed as narrative text (user stories), an outline, a template-based description, or a UML diagram.

Example: A scenario for an online bookstore: the customer logs in, searches for “Software Engineering”, selects a matching book, adds it to the cart, enters shipping/payment details, and receives an order confirmation. The corresponding *use case*, “Purchase a Book”, generalises this one story into a repeatable flow with exceptions (out of stock, payment failure) – see the description table below.

Use-Case Diagram Notation

A use-case diagram is drawn with a fixed vocabulary of symbols: a rectangle for the **system boundary** (labelled with the system name, actors outside it and use cases inside), stick figures for **actors** (external people, devices or systems that interact with the system, representing roles rather than individuals – a **primary actor** directly benefits from the system, a **secondary actor** supports it), ovals for **use cases** (a specific function or service, described from the actor's perspective), solid lines for **association** (an actor interacting with a use case), and dashed arrows with stereotypes for two special use-case relationships: **«include»** (from a base use case to a use case it always executes as part of itself, for common reusable functionality, e.g. “Place Order” includes “Verify Login”) and **«extend»** (from an optional use case to the base use case it conditionally extends, e.g. “Checkout” may be extended by “Apply Coupon”; the base use case is complete without it). **Generalisation** – a solid line with a hollow triangle arrowhead – shows inheritance between actors or between use cases.

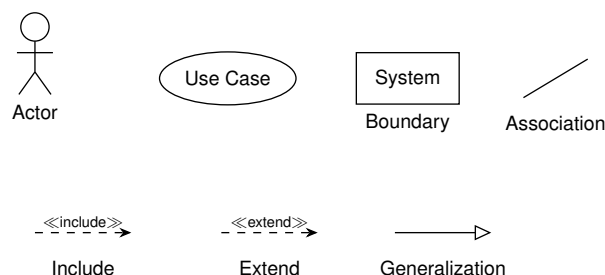


Figure 13: Use-case notation legend: actor, use case, system boundary, association, «include», «extend», and generalization.

Worked Use-Case Diagram 1: Online Food Ordering System

This continues the food-ordering example of Section 3.3: the same functions (browse, cart, pay, track) appear here as use cases with their actors and relationships.

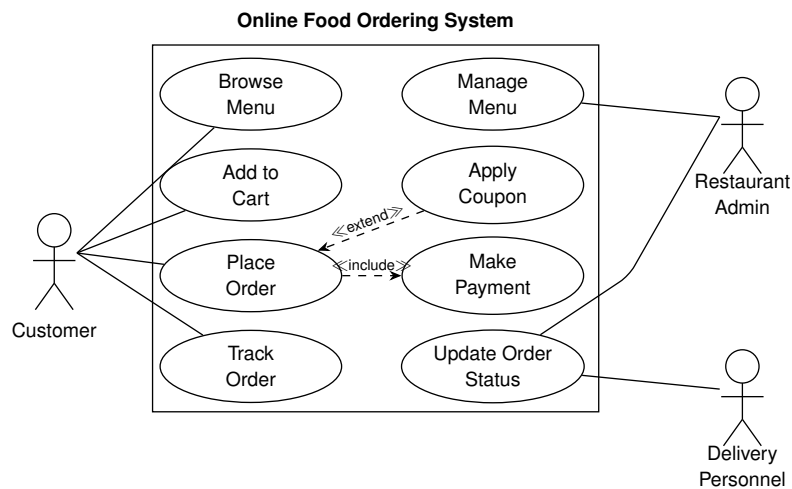


Figure 14: Use-case diagram for an Online Food Ordering System. The Customer browses, carts, places and tracks orders; Restaurant Admin manages the menu and updates order status; Delivery Personnel also updates order status. Placing an order includes making payment; applying a coupon extends placing an order.

Worked Use-Case Diagram 2: Library System

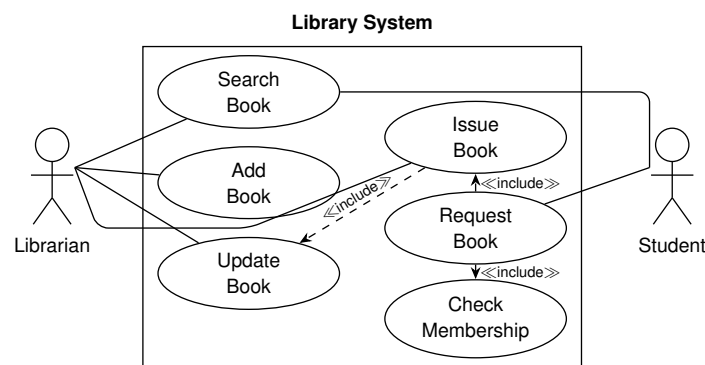


Figure 15: Use-case diagram for a Library System. The Librarian searches, adds and updates books and issues books; the Student searches and requests books. Issuing a book includes updating the book record, and requesting a book includes both issuing it and checking membership.

Use-Case Description: “Purchase a Book”

Once a diagram names a use case, it is written up in a standard template so the flow is unambiguous to designers and testers – **use case name**, **actor(s)**, **precondition**, **trigger**, numbered **main flow**, **alternate/exception flows**, **postcondition**, and **priority** (essential, conditional, optional).

Element	Details
Use Case Name	Purchase a Book
Actor	Customer
Precondition	Customer has a registered account and is logged in.
Trigger	The customer decides to search for and purchase a book.
Main Flow	1. Customer searches for a book. 2. System displays matching results. 3. Customer selects a book. 4. Customer adds the book to the cart. 5. Customer proceeds to checkout. 6. Customer enters shipping and payment details. 7. System validates payment via the payment gateway. 8. System confirms the order and sends a confirmation email.
Alternate / Exception Flows	(1) <i>Out of stock</i> : system displays an “Out of Stock” message and suggests similar books. (2) <i>Payment failure</i> : system notifies the customer and asks them to retry or use a different payment method.
Postcondition	Order is placed, payment is processed, and a confirmation email is sent.
Priority	Essential

3.5 Agile Requirements Engineering

Agile projects capture requirements incrementally, as short, negotiable statements of value rather than an upfront, exhaustive SRS, and refine them continuously as the product and stakeholder understanding evolve.

User Stories and Acceptance Criteria

User story

A **user story** is a short, informal description of a feature told from the perspective of the user who wants the capability. It is a placeholder for conversation, not a rigid specification, and follows the standard format:

“As a [type of user], I want [some goal] so that [some reason/benefit].”

Example: “As a customer, I want to filter products by price range so that I can find items within my budget.”

INVEST criteria for a good user story. **Independent** – self-contained, not dependent on other stories. **Negotiable** – details are open for discussion, not a fixed contract. **Valuable** – delivers clear value to the user or business. **Estimable** – the team can estimate the effort required. **Small** – fits within a single sprint. **Testable** – has clear criteria that allow verification.

Acceptance criteria define the specific conditions a user story must satisfy to be considered complete, bridging the gap between the story and its implementation. They are commonly written in the **Given/When/Then** (Behaviour-Driven Development, BDD) format: **Given** a precondition/context, **When** an action is performed, **Then** an expected result occurs. BDD describes how a system should behave from the user’s or business’s perspective, rather than starting from technical implementation details.

Example: Story: “As a customer, I want to reset my password so that I can regain access if I forget it.”

1. Given the user is on the login page, when they click “Forgot Password” and enter a registered email, then a password reset link shall be sent to that email within 1 minute.
2. Given the user clicks the reset link, when they enter a new password meeting complexity rules, then the password shall be updated and a confirmation displayed.

3. Given the user enters an unregistered email, when they submit the form, then an error message “Email not found” shall be displayed.

Product Backlog Creation and Prioritization

Product backlog

The **product backlog** is an ordered, evolving list of everything needed in the product – features, enhancements and bug fixes. It is the single source of requirements for the development team, and the **Product Owner** is responsible for maintaining and prioritising it.

It is built by gathering initial user stories from stakeholders (interviews, workshops, brainstorming), writing each as a user story with acceptance criteria, estimating effort per story (story points or ideal days), and ordering by priority – the highest-priority items at the top are refined in detail, lower items stay coarser.

Prioritisation techniques. **MoSCoW:** **Must have** (essential, non-negotiable), **Should have** (important, not critical for the current release), **Could have** (desirable if time permits), **Won't have** (explicitly out of scope for now). **Business value vs. effort:** prioritise the highest value relative to development effort. **Risk-based:** address high-risk items early to reduce uncertainty. **Kano model:** classify features as **Basic** (expected), **Performance** (more is better) or **Delighters** (unexpected positive surprises).

Story Mapping Basics

User story mapping

User story mapping (Jeff Patton) is a visual technique for organising user stories to show the “big picture” of the product while keeping the detail of individual stories, addressing the way a flat backlog loses the overall user journey. A **backbone** (horizontal axis) lists the high-level user activities left to right, in the order a user performs them; beneath each activity, **ribs** (vertical axis) stack the specific tasks/stories for it, most important at top; horizontal **release slices** then group stories into releases, with the topmost slice forming the Minimum Viable Product (MVP).

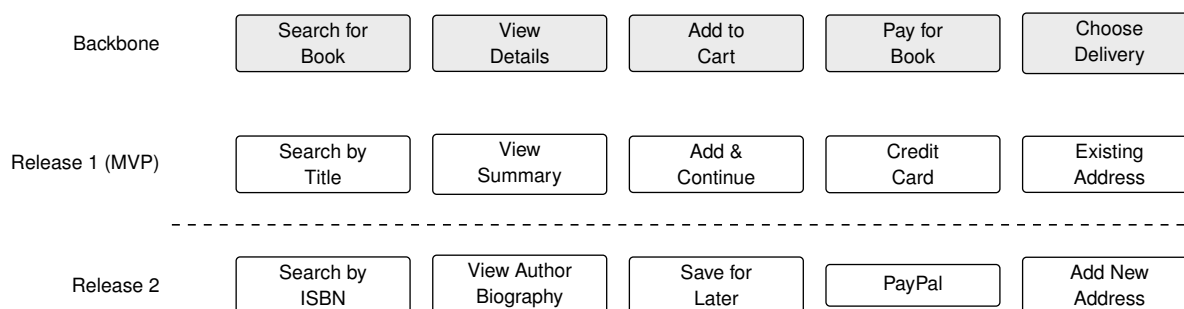


Figure 16: Story map for an online bookstore: backbone activities across the top, with ribs beneath each stacked by priority and sliced into releases – the top slice is the MVP.

Building a story map: frame the problem (identify users and their goals); map the user journey as the backbone; break activities down into tasks/stories as ribs; prioritise stories vertically under each activity; slice horizontally for release planning. Done well, it visualises the entire product scope without losing the big picture, exposes gaps and dependencies, supports release planning by slicing out cohesive, valuable releases, and improves shared understanding between team and stakeholders.

Continuous Requirements Refinement (Backlog Grooming)

Backlog refinement (backlog grooming) is the ongoing process of reviewing, updating and elaborating backlog items so they are **ready** for upcoming sprints – small enough, well understood, estimated,

and carrying clear acceptance criteria – so that sprint planning is quick and the team can start work immediately. It is continuous, not a one-off event: teams typically spend about 10% of sprint capacity on it, in a dedicated mid-sprint meeting or informally throughout, with the Product Owner, development team and Scrum Master participating. Refinement breaks large epics into smaller stories, re-estimates and re-prioritises items as understanding grows, removes obsolete items, adds missing acceptance criteria, and flags dependencies – preparing items for *future* sprints, as distinct from sprint planning, which selects and commits to items for the *next* sprint; without effective refinement, sprint planning becomes chaotic and slow.

Recap. Use cases are drawn as actor(stick figure)–association(line)–use case(oval), inside a labelled system-boundary box, with `«include»`/`«extend»` dashed arrows for reused/optional behaviour. Agile RE replaces the upfront SRS with user stories (“As a . . . , I want . . . , so that . . . ”, judged by INVEST) and Given/When/Then acceptance criteria, organised into a prioritised product backlog (MoSCoW, value/effort, risk, Kano) and visualised as a story map (backbone activities × prioritised ribs × release slices), continuously refined through backlog grooming.

Chapter 4 Architectural Design

4.1 Introduction and Importance

Architectural design

Architectural design is the process of defining the overall structure of a software system: its major components, the relationships and interactions among them, and the constraints that govern their composition. The output is an architectural model showing structural elements (modules, objects, subsystems), their interfaces, the connectors that enable communication, and integration constraints – the “big picture” before component-level design begins.

It is analogous to a building’s floor plan: it shows the layout of rooms (components) and how they connect (interfaces/connectors), without specifying plumbing or wiring detail (component internals).

Why architecture matters. (1) **Communication** – gives developers, managers, customers and testers a common high-level vocabulary. (2) **Early decisions** – style choices (e.g. layered vs. client-server) shape all later data structures, interfaces, control flow and testability; fixing them late is expensive. (3) **Transferable model** – proven styles/patterns are reusable templates for similar systems. (4) **Early quality assessment** – performance, reliability, security and modifiability can be analysed before code exists. (5) **Risk reduction** – structural risks are found and mitigated before they reach the codebase.

Architecture is designed *before* detailed specifications because it is the structural skeleton that specifications must fit; it lets subsystems be identified and allocated to parallel teams, lets non-functional requirements (performance, scalability, security) be assessed while change is still cheap, and – even in agile projects – a lean “walking skeleton” architecture guides sprint planning as requirements continue to evolve.

4.2 Architectural Design Principles

Ten principles (apply regardless of method, style or language):

1. **Traceability** – every element must map back to a requirement, and vice versa.
2. **Architecture first** – settle structure before component-level detail.
3. **Data design matters as much as processing design** – data structure shapes the whole architecture.
4. **Design interfaces (internal & external) with care** – eases integration and testing.
5. **Prioritise UI ease of use** – a poor UI makes even good architecture look “bad”.
6. **Components should be functionally independent** (cohesive, single well-defined function).
7. **Components should be loosely coupled** – minimise inter-component dependency.
8. **Design representations must be understandable** – an effective medium for coders, testers, maintainers.
9. **Develop the design iteratively** – each pass refines, corrects, simplifies.
10. **Agile projects still need architecture** – a lean, code-synced design model conveys purpose and inter-module interaction that source code alone cannot.

4.3 Taxonomy of Architectural Styles

Architectural style

An **architectural style** defines a family of systems by a pattern of structural organisation: (1) a set of components performing system functions, (2) a set of connectors enabling communication/coordination, (3) constraints on how components integrate, and (4) semantic

models describing overall system properties.

1. Data-Centered (Repository) Architecture

A central data store (database, file or shared repository) is accessed by independent components, which never communicate directly with one another – only through the store. **Passive** repositories just serve data on request; in the **blackboard** variant the repository is active and notifies clients when data of interest changes (used for non-deterministic problems, e.g. speech recognition). **Adv**: components added/changed independently; centralised backup, security, consistency. **Disadv**: repository is a single point of failure; components tightly coupled to the data schema; hard to distribute. *Example*: A CASE tool suite (editor, compiler, debugger, VCS) sharing one project repository.

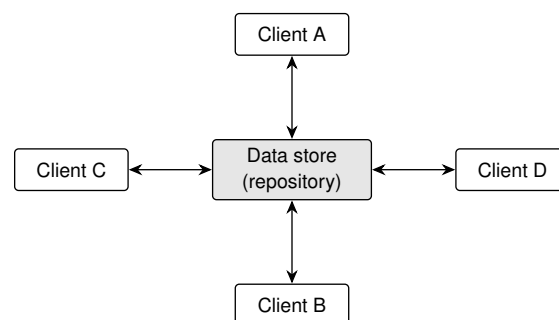


Figure 17: Repository style: components interact only through the central data store.

2. Data-Flow (Pipe-and-Filter) Architecture

Input data is transformed by a series of independent **filters** connected by **pipes**, each filter reading input in a defined format and writing output in a defined format, without knowing a neighbour's internals. **Adv**: filters are reusable and recombinaable; easy to add/replace a filter; supports concurrency. **Disadv**: unsuited to interactive applications; transformation overhead; awkward for complex branching control. *Example*: A Unix pipeline: `cat file.txt | grep "error" | sort | uniq -c`.



Figure 18: Pipe-and-filter style: data flows through independent filters via pipes.

3. Call-and-Return Architecture

A “main” program invokes subprograms, which may invoke further subprograms, forming a control hierarchy. **Main program/subprogram**: classic hierarchical decomposition on one machine. **Remote Procedure Call (RPC)**: the same hierarchy but components are distributed across networked machines. **Adv**: easy to modify/scale; well understood; clear control flow. **Disadv**: can be rigid – hierarchy changes force restructuring; tightly coupled control flow reduces flexibility.

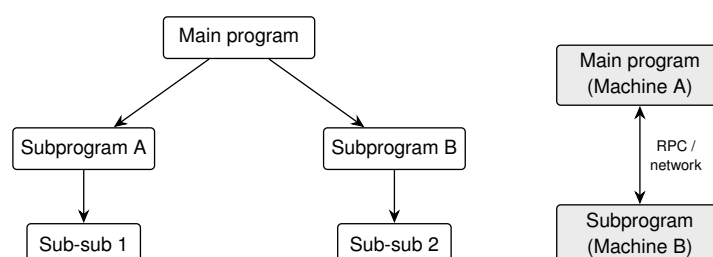


Figure 19: (a) Main program/subprogram hierarchy; (b) RPC – the same hierarchy, distributed.

4. Object-Oriented Architecture

Components encapsulate both data and the operations on it; components communicate by **message passing**, each object managing its own state behind a well-defined interface. **Adv:** promotes reuse, modularity, information hiding; an object's internal changes do not affect others; maps naturally to real-world domains. **Disadv:** overall structure harder to visualise; message-passing overhead; complex interdependencies if designed carelessly.

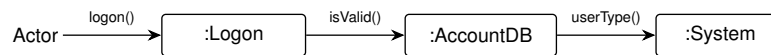


Figure 20: Object-oriented style: objects collaborate by passing messages (login scenario).

5. Layered Architecture

The system is organised into horizontal layers, each providing services to the layer above and using services of the layer below; outer layers are closer to the user, inner layers closer to the hardware. **Adv:** clear separation of concerns; layers developed / tested / modified independently; a layer can change without affecting others if its interface is preserved; supports reuse. **Disadv:** requests may traverse many layers (overhead); not every system decomposes cleanly into layers. *Example:* An OS: UI (shell/GUI) → system utilities (file, process mgmt) → kernel (memory, CPU scheduling) → hardware-abstraction (device drivers).

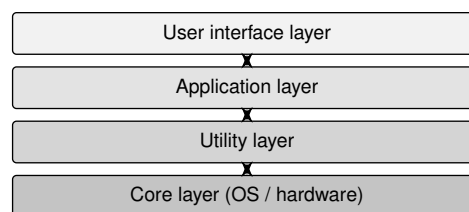


Figure 21: Layered style: each layer serves the one above and uses the one below.

6. Client-Server Architecture

Work is partitioned between **servers** (host shared resources/services, listen for requests) and **clients** (present the UI, send requests over a network). **Adv:** centralised data/security management; scalable (upgrade or add servers); resource-efficient thin clients; data integrity kept at the server. **Disadv:** server is a single point of failure; depends entirely on network connectivity; server can bottleneck under load; higher infrastructure cost. *Example:* Online banking: the mobile app (client) sends transaction requests to the bank's application server, which queries the database server and returns results.

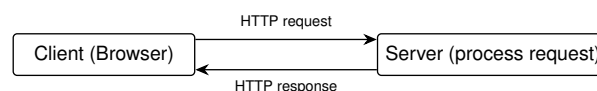


Figure 22: Client-server style: clients request; the server processes and responds.

7. Multi-Tier Architecture (2-Tier and 3-Tier)

An extension of client-server that separates functionality into distinct tiers on separate infrastructure. **2-tier:** the client (UI + some application logic) talks directly to the server (data storage + remaining logic) – e.g. a desktop app connecting straight to a database server. **3-tier** inserts a middle tier: **presentation** (client UI), **application/logic** (business rules), **data** (database) – e.g. an e-commerce browser → web/application server → database server. **Adv of 3-tier over 2-tier:** better separation of concerns; the logic tier scales independently; the database is not directly exposed (more secure); logic changes do not touch the client or database.

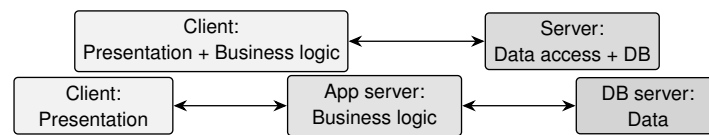


Figure 23: (a) 2-tier: client talks directly to the server; (b) 3-tier: an application tier is inserted between client and database.

8. Model-View-Controller (MVC) Architecture

Separates an application into three parts: the **Model** (data, business logic, state), the **View** (renders data to the user) and the **Controller** (interprets user input, invokes the Model, selects the View). Flow: user acts on the View → Controller invokes the Model → Model returns results → Controller selects a View → View renders the response. **Adv:** UI and business-logic work proceed independently; supports multiple views of the same data; the Model can be tested in isolation. **Disadv:** adds complexity for simple applications; Controller-Model coupling can grow tight if not designed carefully.

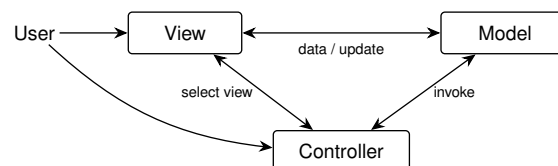


Figure 24: MVC: the Controller mediates between View and Model; View and Model also interact directly.

Repository Model	Layered Model
Independent components interact only through a central data store.	Horizontal layers, each serving the layer above and using the layer below.
Efficient for systems with large shared data sets.	Efficient for systems with clear functional separation; data crosses layers (overhead).
Components are coupled to the data model, not to each other.	Layers are coupled only through well-defined interfaces.
Adding/removing components is easy; changing the schema is hard.	Changing a layer is easy as long as its interface is preserved.
The data store is a single point of failure.	No single point of failure unless a critical layer fails.
Best for CASE tools, IDEs, data warehouses.	Best for web applications, operating systems, network protocols.

4.4 Modular Design, Cohesion and Coupling

Modularity

Modularity – the chief expression of **separation of concerns** in design – divides software into separately named, addressable **modules** that integrate to satisfy system requirements. A monolithic (single-module) program is nearly impossible to understand, test or maintain; modularisation makes it intellectually manageable, and eases planning, incremental delivery, change, testing and maintenance.

As the number of modules rises, cost per module falls (smaller, simpler modules) but integration cost rises (more interfaces); total cost is therefore U-shaped, with an optimal module count M minimising it – too few modules leaves each too complex, too many causes excessive integration overhead.

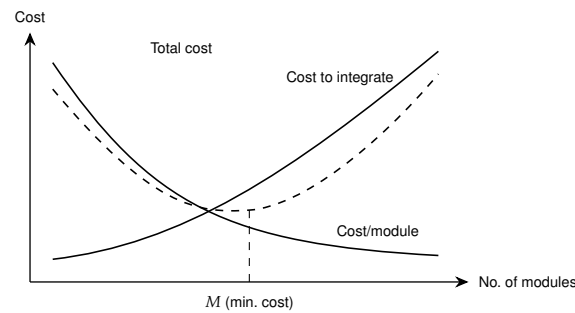


Figure 25: Modularity–cost trade-off: total cost is minimised at an optimal module count M .

Two decomposition styles are commonly used: **functional (top-down) decomposition** – break a high-level function into sub-functions, repeatedly, until each module does one well-defined task (*Example:* order processing → Validate Order, Calculate Total, Process Payment, Generate Invoice); and **object-oriented decomposition** – decompose into objects/classes mapping to real-world entities (*Example:* an e-commerce system’s Customer, Product, ShoppingCart, Order, Payment).

Information hiding (Parnas)

Modules should be designed so internal details – algorithms, data structures, design decisions – are hidden from other modules; only the module’s **interface** (what it does, not how) is visible. This limits how far an error introduced by a change can propagate, and pays off most during testing and maintenance.

A module is **functionally independent** if it performs a single well-defined function and interacts with others only through simple, well-defined interfaces – an outgrowth of separation of concerns, modularity, abstraction and information hiding. Independent modules are easier to develop, test, maintain and reuse. Functional independence is assessed by two qualitative measures: **cohesion** (strength of relatedness *within* a module) and **coupling** (degree of interdependence *between* modules).

Cohesion (worst → best)

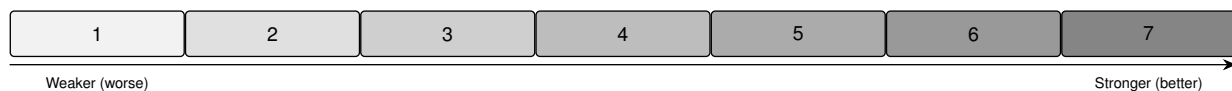
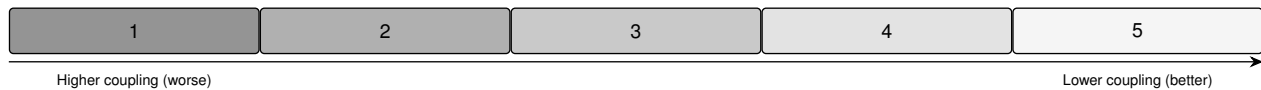


Figure 26: Cohesion spectrum, worst (1) to best (7) – numbers key the table below.

#	Cohesion type	Example
1	Coincidental	A “utility” module with unrelated <code>printReport()</code> , <code>calculateTax()</code> , <code>validateEmail()</code> .
2	Logical	One input module handling keyboard, file and network input via a control flag.
3	Temporal	<code>startup()</code> that inits the DB connection, loads config, sets the clock – grouped only by timing.
4	Procedural	Read a record, validate it, write it – related only by execution sequence.
5	Communicational	A report module reading customer data, totalling, and formatting – all share the same data.
6	Sequential	<code>parseInput()</code> output feeds <code>validateData()</code> , whose output feeds <code>transformData()</code> .
7	Functional	<code>computeSquareRoot()</code> – every element serves one single, well-defined function.

Guidance: always aim for functional cohesion; communicational and sequential are acceptable; avoid coincidental, logical and temporal.

Coupling (worst → best)**Figure 27:** Coupling spectrum, worst (1) to best (5) – numbers key the table below.

#	Coupling type	Example
1	Content	Module A directly reads/modifies a variable inside Module B – violates information hiding.
2	Common	Several modules read/write the same global configuration array.
3	Control	Module A calls <code>processRecord(flag)</code> ; the flag tells B whether to insert, update or delete.
4	Stamp	A whole Customer object is passed to a module that only needs the email field.
5	Data	<code>calculateArea(length, width)</code> – only the required primitive values are passed.

Guidance: always aim for data coupling; avoid content and common coupling; minimise control coupling by redesigning interfaces.

High cohesion, low coupling. Cohesion and coupling are inversely related – raising cohesion within modules tends to lower coupling between them. A system with **high cohesion** and **low coupling** is easier to understand, test, maintain and reuse: this is the hallmark of a well-designed modular system, and the goal of Principles 6–7 (§4.2).

Chapter 5 System Modeling

5.1 System Modeling and Abstraction

Need for System Modeling

System modeling

System modeling is the process of developing abstract representations (models) of a system. Each model presents a different view of the system – its data, its functions, its behaviour, or its structure – typically expressed in a standardised graphical notation such as UML. Models are used during requirements analysis to understand the existing system and the requirements for the new one, and during design to describe the system to the engineers who will build it.

Because models are abstractions, they deliberately omit detail, so stakeholders can focus on one aspect of the system without being overwhelmed by its full complexity. Modeling matters for five reasons: it **clarifies understanding** by forcing stakeholders to think carefully about requirements, exposing ambiguities and gaps that prose hides; it **facilitates communication**, since a well-drawn diagram is a common language between developers, customers, managers and testers; it **bridges requirements to design**, giving traceability from customer needs through architecture to implementation; it **supports analysis and validation**, letting a model be checked against requirements before any code is written, when errors are cheapest to fix; and it **documents the system** for later maintenance and evolution.

Three objectives of the requirements model. (1) Describe what the customer requires; (2) establish a basis for creating the software design; (3) define a set of requirements that can be validated once the software is built.

Role of Abstractions in Managing Complexity

Abstraction

Abstraction is the fundamental mechanism used to manage complexity in system modeling. A real system may involve thousands of interacting users, devices, data items, rules and functions; modeling all of it at once is impossible, so abstraction deliberately hides what is irrelevant to the current purpose and keeps only what matters for that view.

Abstraction manages complexity in four ways. It **hides unnecessary detail** – a context diagram hides internal processes, a class diagram hides algorithmic detail. It **separates concerns** – use cases capture functional interaction, class diagrams capture data structure, activity diagrams capture flow of control, so no single model tries to capture everything at once. It **enables hierarchical decomposition** – a single high-level abstraction (“the entire system” in a context diagram) is progressively refined into finer-grained components, so each piece can be managed independently. And it **supports multiple viewpoints** – a customer needs a use-case view, an architect a structural view, a tester a behavioural view, and each is a separate, complementary model of the same underlying system.

Five principles of requirements modeling. (1) The information domain (data flowing in, out and stored within the system) must be represented; (2) the functions the software performs, user-visible and internal, must be defined; (3) the software’s behaviour as a consequence of external events must be represented; (4) models of information, function and behaviour must be partitioned hierarchically, uncovering detail in layers; (5) analysis should move from essential (problem-domain) information toward implementation detail.

5.2 Process Modeling Using DFD

Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** is a graphical representation that models how data moves through a system – its inputs, outputs, transformations (processes) and stores of data – without prescribing implementation detail. DFDs are a core technique of structured analysis: they focus on *what* the system does with data, not *how* it does it.

Notation

The course uses the Yourdon–DeMarco symbol set throughout.



Figure 28: DFD notation. A process transforms incoming flows into outgoing flows and is named with a verb; a data store is data at rest, named with a noun; an external entity is a source/sink outside the system boundary; a data flow is a named, directed arrow.

Levels of Abstraction

DFDs are organised hierarchically, each level showing progressively more decomposition.

- **Context diagram (Level-0)** – the whole system as a single process, with external entities around it and labeled data flows between them; no internal processes or stores are shown. It fixes the system boundary: what is inside vs. outside.
- **Level-1 DFD** – decomposes the single process into its major sub-processes, showing the data flows between them, the data stores they use, and the same external entities.
- **Level-2+ DFD** – further decomposes an individual Level-1 process into finer sub-processes; not always needed, and decomposition stops once each process is simple enough to describe in a short process specification (minispec).

Every level must obey the **balancing rule**: every data flow entering or leaving a parent process must also appear, in total, in its child (decomposed) diagram – the parent and child diagrams must be consistent with each other.

Rules for drawing DFDs. Every process must have at least one input and at least one output (no “black holes” or “miracles”). Data cannot flow directly between two external entities, or between two data stores, or between an external entity and a data store – it must always pass through a process. Every process, data flow, data store and external entity must be named. Parent and child diagrams must balance (the DFD balancing rule above).

Worked Example 1: Food Ordering System

Example: a Food Ordering System (FOS) takes a customer’s order, sends it to the kitchen, bills the customer, and reports to the manager.

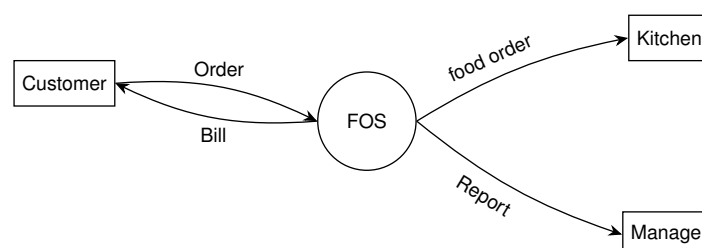


Figure 29: Food Ordering System – context diagram (Level-0).

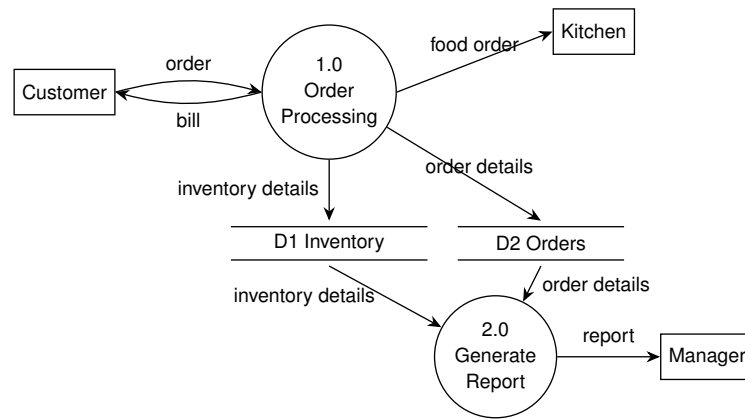


Figure 30: Food Ordering System – Level-1 DFD, decomposing FOS into Order Processing (1.0) and Generate Report (2.0).

Process 1.0 can be decomposed once more into a Level-2 diagram, refining it into 1.1 Receive Order and 1.2 Generate Bill:

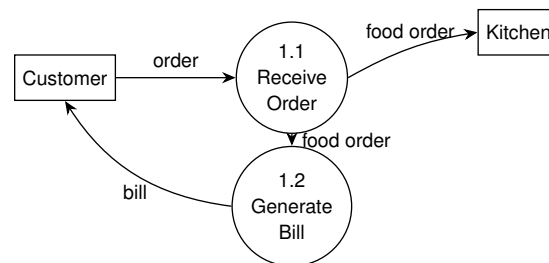


Figure 31: Food Ordering System – Level-2 decomposition of process 1.0 (Order Processing).

Worked Example 2: Library Management System

Example: a Library Management System lets a user search for and borrow/return books, and lets a librarian maintain the catalogue, manage users and pull reports.

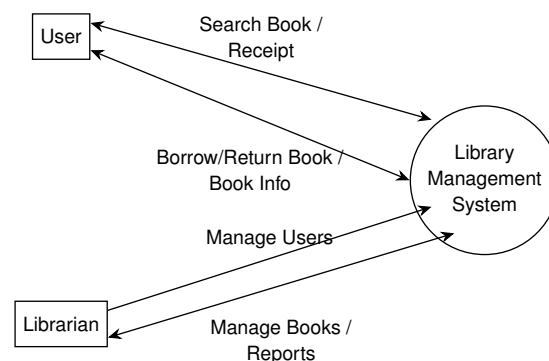


Figure 32: Library Management System – context diagram (Level-0). Double-headed arrows mark a paired request/response flow between the same two entities.

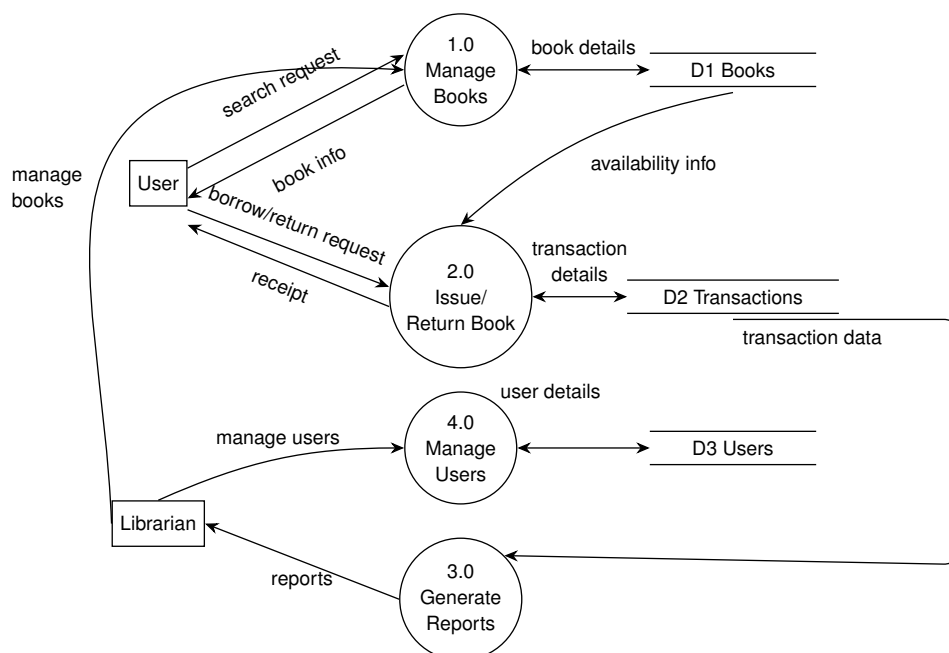


Figure 33: Library Management System – Level-1 DFD. Double-headed arrows mark a read/write pair with the same store.

5.3 Scenario-Based Analysis

Concept of Scenarios

Scenario

A **scenario** is a specific sequence of actions and interactions between an actor and the system that accomplishes a particular goal. Scenarios describe the system from the user's point of view and answer the question "how will the system be used?". The primary tool for capturing a scenario is the **use case**, which describes one usage scenario as a "contract for behaviour" in plain language, from the perspective of a defined actor.

Use cases are often the first requirements model to be developed, since they directly capture user expectations. Scenarios serve several purposes: they let stakeholders validate that the system will meet their needs, give developers clear functional expectations, drive the identification of classes and objects for later design, and form the basis for test-case development.

Use-Case Descriptions and Diagrams

Use-case descriptions (actor, precondition, main flow, alternative flow, postcondition) and use-case diagrams (actors as stick figures, use cases as ellipses, a system boundary box, association/include/extend relationships) are the detailed, diagrammed form of a scenario. Worked examples of both are given with Section 3.4 (Requirements Engineering), where they are developed directly from a system's functional requirements; they are not repeated here to avoid duplicating that material.

5.4 Behavioral and Structural Modeling

Activity Diagrams

Activity diagram

An **activity diagram** is a UML behavioral model that shows the flow of control from one activity to another within a process – the sequence of steps, the decisions that branch the flow, and the points where work proceeds in parallel. Where a DFD shows data moving through processes, an activity diagram shows the *order in which work happens*, including concurrency and synchronisation.

Activity diagrams work at two levels: at the business-process level they model a workflow spanning several people or systems, such as the order-fulfilment example below; at the design level the same notation models the internal logic of a single operation (its algorithm), one action per statement or block.

Notation

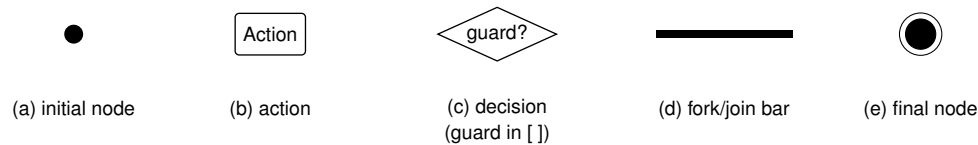


Figure 34: Activity diagram notation. A solid dot starts the flow; a rounded box is one action; a diamond branches the flow, each outgoing arrow labeled with a guard condition in square brackets; a filled bar forks one flow into several concurrent flows (fork) or synchronises several concurrent flows back into one (join); a bullseye ends the flow.

A **fork** (one incoming flow, several outgoing) starts concurrent activities; the matching **join** (several incoming, one outgoing) does not release its outgoing flow until *all* incoming branches have completed – it is a synchronisation point, not just a merge. A plain **decision/merge** diamond, by contrast, only ever carries one flow at a time down one of its branches.

Worked Example: Process Order

Example: an order-fulfilment process that must arrange delivery (rushed or regular) *and*, concurrently, send an invoice and collect payment, before the order is considered complete.

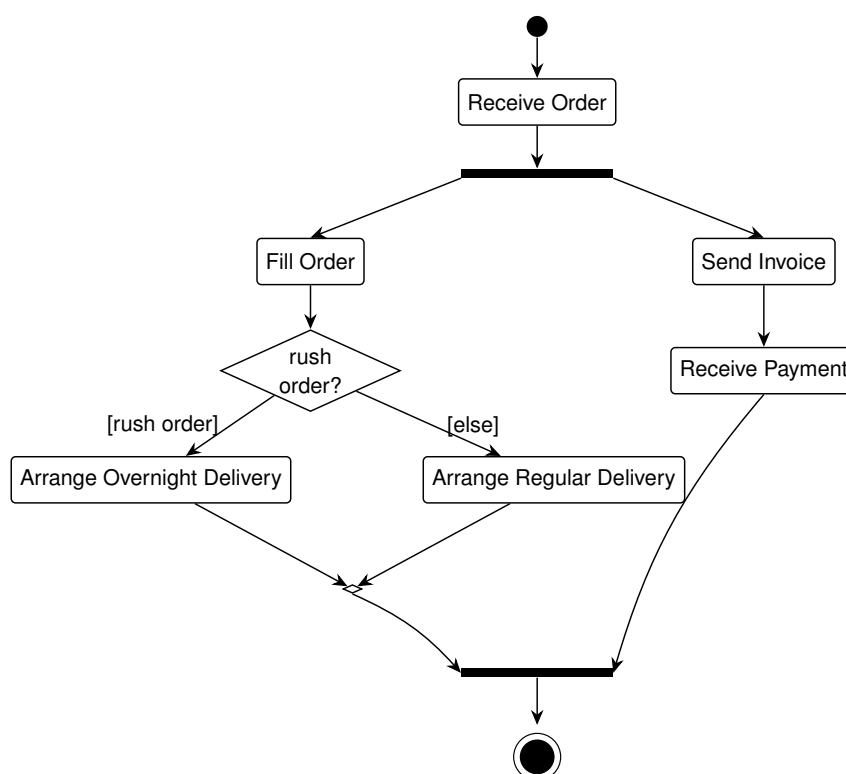


Figure 35: Activity diagram for *Process Order*. After Receive Order, a fork splits the flow into two concurrent branches: Fill Order (with a rush/regular delivery decision, remerging at the diamond) and Send Invoice → Receive Payment. The join bar waits for *both* branches to finish before the order reaches its final node.

A **swimlane diagram** (activity partition) is the same notation redrawn inside vertical or horizontal bands, one per responsible actor (e.g. Customer, Order System, Warehouse), so that each action is

placed in the lane of whoever performs it and a lane-crossing arrow marks a handoff between actors – it answers “who does what?” in addition to “what happens next?”.

Class-Based Modeling

Class diagram

Class-based modeling identifies the classes (things) a system manipulates, the **attributes** that describe each class, the **operations** that act on it, and the **relationships** between classes. Candidate classes are usually found by a **grammatical parse** of the use-case descriptions – underlining the nouns and noun phrases – then filtered against criteria such as whether the class needs to be remembered, has its own operations, and has more than one attribute. The result is drawn as a **UML class diagram**.

Identifying Analysis Classes

Candidate classes are found by a **grammatical parse**: underline every noun and noun phrase in the use-case descriptions and processing narratives, list them as potential classes, then eliminate duplicates, synonyms, and nouns that are really attributes of another class.

Selection criteria for a class (Coad and Yourdon). **Retained information** – the system must remember something about it to function; **needed services** – it has operations that change its own attribute values; **multiple attributes** – a class with only one attribute is usually just an attribute of something else; **common attributes / common operations** – a set of attributes or operations applies to every instance of it; **essential requirements** – external entities that produce or consume information essential to the system are almost always modeled as classes.

Analysis classes typically fall into seven categories: external entities, things (reports, displays, signals), occurrences/events, roles, organisational units, places, and structures. Once a class is confirmed, its **attributes** are found by asking “what data items fully define this class?” and its **operations** by extracting verbs from the use cases; operations generally fall into four kinds – data manipulation, computation, state inquiry, and event monitoring.

Notation

Each class is a rectangle in three compartments: name, attributes, operations. Visibility is marked - private, # protected, + public.

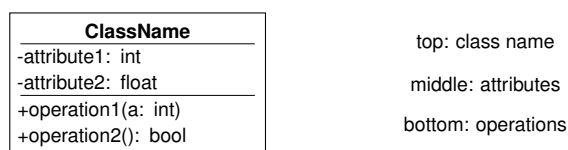


Figure 36: Generic class-box notation: three compartments for name, attributes and operations.

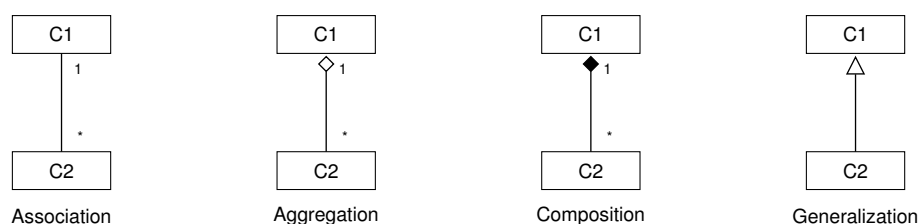


Figure 37: Relationship notation. **Association**: a plain line, optionally with a role name and multiplicities. **Aggregation**: hollow diamond at the “whole” end – the part can exist without the whole. **Composition**: filled diamond at the “whole” end – the part is destroyed with the whole. **Generalization**: hollow triangle pointing from subclass (C2) to superclass (C1).

Multiplicity states how many instances of one class relate to a single instance of another, written at the line's ends: 1 exactly one, 0..1 zero or one, 1..* one or more, 0..* (or *) zero or more.

Worked Example: Banking System

Example: a bank that owns branches, each branch holds accounts, accounts specialise into savings and current accounts, and each such account belongs to one customer.

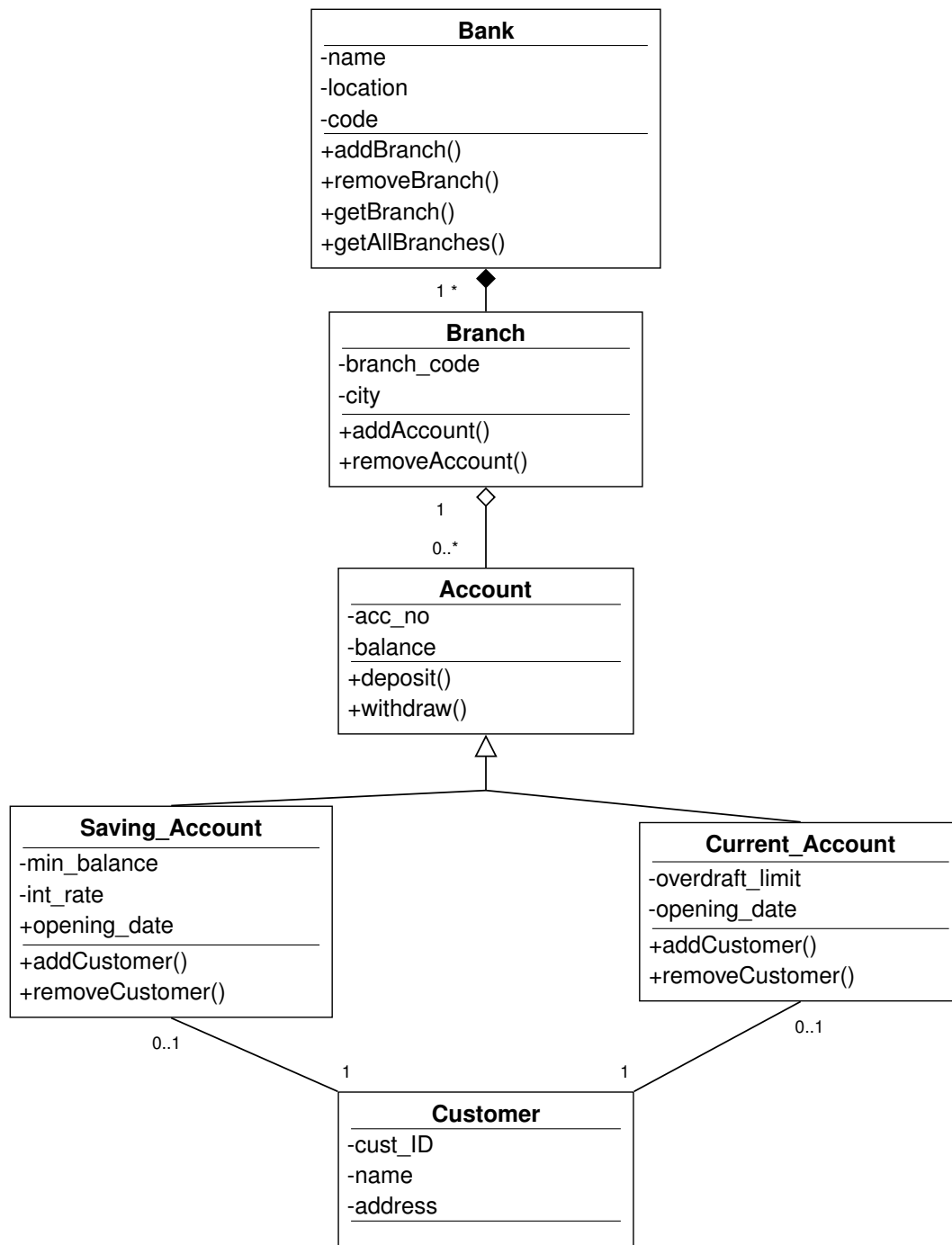


Figure 38: Class diagram for a Banking system. Bank is composed of Branches (1 Bank to * Branch – a branch cannot exist without its bank); a Branch aggregates Accounts (1 to 0..* – an account could in principle be reassigned); Saving_Account and Current_Account generalise (inherit) from Account; each is associated with exactly one Customer, while a Customer may hold 0..1 of each account type.

CRC Cards

CRC card

A **CRC (Class-Responsibility-Collaborator) card** is a lightweight, index-card-sized alternative to a full class diagram: for each class it lists only the class name, its **responsibilities** (the attributes it must know and the operations it must perform) and its **collaborators** (other classes it needs to fulfil those responsibilities). CRC cards are quick to sketch on paper, making them well suited to brainstorming classes early in analysis and to **role-playing reviews**, where each reviewer holds a card and walks through a use case to check that every responsibility is covered and every collaboration actually works.

Example: the ShoppingCart class from an online e-commerce system, which manages the items a customer has selected before checkout.

Responsibility	Collaborator
Adds a product and tracks its quantity	Product
Removes a product from the current selection	Product
Calculates the subtotal cost of all items inside	Product
Empties all contents after a successful checkout	Order

Table 1: CRC card – ShoppingCart (Online E-Commerce Shopping System).

Chapter 6 Coding and Testing

6.1 Coding Standards and Guidelines

Coding standards

Coding standards are a set of rules and conventions that a development team agrees to follow when writing source code: how it is structured, named, formatted and documented. The purpose is code that is consistent, readable, maintainable and less error-prone across the whole team, regardless of which individual wrote a given piece of it.

They matter because: (1) **readability** – any team member, not just the author, can follow the code; (2) **maintainability** – since 60–80% of total software effort is spent on maintenance, uniform conventions are markedly cheaper to modify and extend; (3) **reduced errors** – standards enforce good practice (proper initialisation, boundary checking); (4) **easier reviews** – reviewers focus on logic, not style disputes; (5) **faster onboarding** – new members read a predictable codebase faster.

Areas covered by coding standards.

1. **Naming conventions** – camelCase for variables/functions (`totalPrice`, `calculateTax()`); PascalCase for classes (`OrderManager`); UPPER_SNAKE_CASE for constants (`MAX_RETRIES`). Names should be self-descriptive.
2. **Indentation and formatting** – a fixed indent width (2 or 4 spaces), a line-length limit (80–120 chars), one agreed brace/whitespace style, so control structure is visually obvious.
3. **Commenting and documentation** – comments explain *why*, not *what* (the code itself should say what); every file/module carries a header (purpose, author, date); avoid restating the obvious.
4. **Code organisation** – group related functions logically; keep a consistent project/directory structure; remove dead code.
5. **Error handling** – every function handles error conditions explicitly; return values are checked; exceptions follow one project-wide pattern.

Standards vs. guidelines: standards are strict rules that *must* be followed (“all class names use PascalCase”); guidelines are recommended best practice allowing flexibility (“prefer composition over inheritance where reasonable”). Both raise quality, but only standards are mandatory and enforceable.

6.2 Code Review

Code review

Code review is the systematic examination of source code by peers, to find errors, improve quality and share knowledge. It is a form of **static testing** – the code is analysed without being executed. Reviews are among the most effective techniques for early defect detection; industry studies report review techniques up to 75% effective at uncovering design flaws.

An error found at review is far cheaper to fix than the same error found later: a requirements error found at review may cost about 6 person-hours to fix, versus roughly 45 person-hours if it is only caught during testing. Left uncorrected, an early error **amplifies** – one requirements defect can seed several design defects, which in turn seed still more coding defects; review at each stage breaks this amplification chain. Reviews also catch what testing often misses (logic errors, inefficient algorithms, security flaws, standards violations) and spread team knowledge.

Code Walkthrough

Code walkthrough is an informal, author-led peer review: the developer organises the session and walks a small group (typically 2–5 peers) through the code line-by-line or function-by-function,

explaining logic and design choices. Discussion is free-form; notes are informal and there is no formal follow-up. It requires little advance preparation, is best for knowledge-sharing and quick feedback, and catches high-level design problems well – but is less effective than a formal inspection at finding subtle defects, since reviewers have not studied the code beforehand.

Code Inspection

Code inspection (a **Fagan inspection**, after Michael Fagan, who formalised it at IBM in 1976) is a highly formal, structured peer review aimed specifically at systematic defect detection. Defined roles – **moderator** (leads, not the author), **reader** (presents the code, not the author), **recorder** (logs every defect) and **author** (answers questions but does not lead) – keep it focused on finding defects, not solving them; solutions are explicitly *not* discussed in the meeting.

The six phases of a Fagan inspection.

1. **Planning** – the inspection leader selects the review team (3–5 people), distributes materials, sets the schedule.
2. **Overview** (optional) – the author gives the team background on the work product.
3. **Preparation** – each reviewer independently studies the code against predefined checklists, noting possible defects (typically 1–2 hours/reviewer).
4. **Inspection meeting** – the moderator runs a time-boxed session (under 2 hours); the reader walks through the code; the recorder logs every defect found.
5. **Rework** – the author corrects all identified defects.
6. **Follow-up** – the moderator verifies every defect was properly corrected.

The team's outcome is to **accept** the product as-is, **accept provisionally** with minor fixes (no re-inspection needed), or **reject** for severe errors (re-inspection required after correction). **Faults inspection is good at:** logic errors, missing/incorrect functionality, standards violations, data-type mismatches, bad initialisation, boundary-condition errors, unreachable code, wrong interface usage, security flaws. **Faults inspection tends to miss:** real-load performance bottlenecks, timing/concurrency issues, hardware-dependent failures, usability problems, and defects that only surface from complex multi-module run-time interaction.

FTR (formal technical review) guidelines. Review the product, not the producer – stay constructive. Set and keep an agenda; limit debate and rebuttal (park issues for later). Point out problems, but do not solve them in the meeting. Limit attendees and insist on advance preparation. Use a checklist per work-product type. Budget dedicated time and resources for reviews.

Cleanroom Technique

Cleanroom software engineering, developed by Harlan Mills at IBM (named after the hardware “cleanroom”, where contamination is *prevented* rather than cleaned up), emphasises **defect prevention over defect removal**: (1) **no developer unit testing** – developers never compile/execute their own code to debug it, forcing more careful reasoning; (2) **formal verification** – each code segment is shown, by mathematical/logical reasoning, to be consistent with its specification, instead of being tested; (3) **incremental development** – small increments are each specified, designed, verified, then handed to an independent test team; (4) **statistical usage testing** – an independent team tests each increment with cases generated from the software's *expected usage profile*, so results are statistically representative of real use; (5) **statistical quality certification** – usage-test results feed a reliability model to compute mean time to failure (MTTF); an increment is certified if it meets its reliability target, else returned for correction.

Aspect	Walkthrough	Fagan inspection	Cleanroom
Formality	Informal	Highly formal, defined roles	Highly formal, verification-based
Led by	The author	A trained moderator (not the author)	No review meeting; formal correctness proof by the developer
Developer testing	Author still unit-tests own code	Author still unit-tests own code	No developer unit testing at all
Primary goal	Knowledge-sharing, quick feedback	Systematic, measured defect detection	Prevent defects; certify reliability statistically

6.3 Software Testing Fundamentals

Software testing

Software testing is the process of executing a program with the intent of finding errors. A good test has a high probability of revealing an as-yet-undiscovered error. Testing can only demonstrate the *presence* of defects, never their absence, and exhaustive testing (every possible input/path) is practically impossible for any non-trivial program.

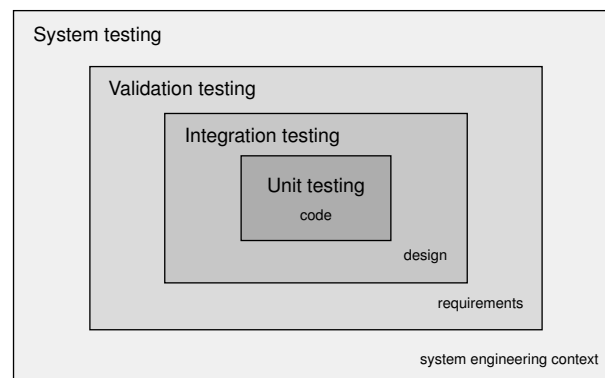


Figure 39: Testing strategy: successive levels widen outward from the code (unit testing) to the whole system in its operating context (system testing), each level corresponding to one development phase.

Verification and Validation

Verification vs. validation

Verification – “*Are we building the product right?*” It checks that each phase’s output correctly implements that phase’s input specification (technical reviews, inspections, walkthroughs, model analysis). **Validation** – “*Are we building the right product?*” It checks that the software meets the customer’s actual needs (acceptance testing, usability testing, customer demonstrations). V&V planning matters because it fixes what is tested, when and by whom (preventing ad hoc testing), allocates test effort by risk, sets pass/exit criteria per phase, and traces every requirement to a test case.

Unit Testing

Unit testing exercises the smallest testable unit – typically one function, method or component – in isolation, to verify it performs its intended job correctly. It checks: the **interface** (data flows in/out correctly – parameter types, order, count), **local data structures** (variables keep integrity through the algorithm), **independent paths** (every execution path is exercised at least once), **boundary conditions** (behaviour at input/output limits), and **error-handling paths** (errors are caught and reported properly). Because a unit is not a standalone program, testing it needs scaffolding: a **driver** is a small “main program” that calls the unit, feeds it test data, and collects results; a **stub** (“dummy subprogram”)

replaces a module the unit-under-test calls – it matches the real interface, does minimal work, and returns control.

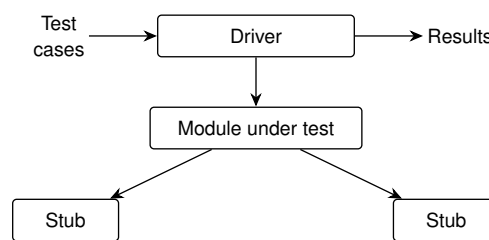


Figure 40: Unit-test scaffolding: a driver supplies test data and collects results; stubs stand in for the modules the unit under test calls.

Unit testing alone is not sufficient: tested in isolation, a unit can still fail once combined with others – interface mismatches, integration problems, system-level performance issues and multi-unit interaction defects are all invisible to it.

Integration Testing

Integration testing systematically combines unit-tested components and tests them as a group, focused on errors in the *interfaces and interactions* between them. Even if every unit works correctly alone, combining them can lose data across an interface, let one component have unintended side effects on another, fail to produce the desired combined function, or let individually acceptable rounding/imprecision accumulate past tolerance.

Integration strategies. Big-bang: combine every component at once and test the whole system together. Discouraged – when an error appears, isolating its cause in the whole program is very hard. **Top-down:** start from the main control module; add subordinate modules one at a time (depth-first or breadth-first); **stubs** replace modules not yet integrated. Tests major control/decision points early, but stubs can become complex. **Bottom-up:** start from the lowest-level (atomic) modules, combined into clusters; **drivers** coordinate each cluster and are discarded as integration moves upward. Avoids complex stubs, but the complete system is not visible until the last module is added. **Sandwich/hybrid:** combines top-down and bottom-up – upper layers integrated top-down, lower layers bottom-up, meeting in the middle – to get both strategies’ advantages. **Continuous integration:** components are merged into the evolving build one or more times a day, each merge triggering automated tests; standard in agile/DevOps. **Smoke testing** rebuilds and tests the build daily to confirm it is stable enough for further testing.

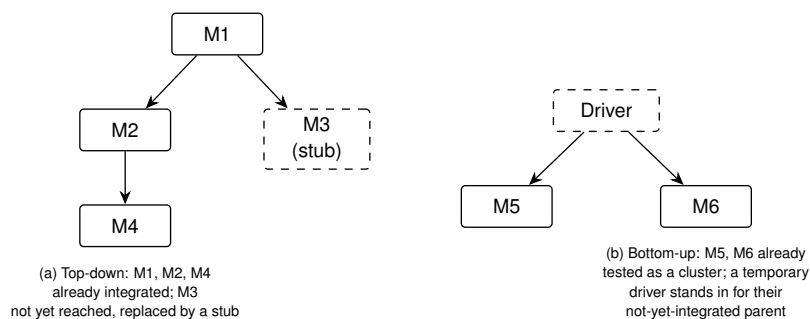


Figure 41: Solid = already-integrated real modules; dashed = temporary scaffolding (stub or driver) standing in for a module not yet integrated.

After each integration step, **regression testing** re-runs a subset of previously passed tests to confirm the new addition has not broken existing functionality; the regression suite mixes tests covering all functions, tests focused on functions likely affected by the change, and tests targeting the changed components directly. Both stubs and drivers are pure testing overhead – written to enable testing, but

never shipped with the final product.

System Testing

System testing tests the fully integrated software as a whole, within the larger computer-based system (hardware, databases, networks, people), verifying it meets all functional, behavioural and performance requirements together. It differs from integration testing, which checks only the *interfaces* between combined components – system testing validates the *entire* system against the original requirements.

Types of system testing. **Recovery testing** – forces failure and checks proper recovery (auto-restart, data integrity after a crash). **Security testing** – attempts to breach protection mechanisms. **Stress testing** – pushes the system beyond normal limits to see how/when it degrades or fails. **Performance testing** – measures response time, throughput and resource use under expected and peak load.

Acceptance Testing

Acceptance testing is run by the end users/customer to decide whether the software meets their business requirements and is ready for deployment – the final testing phase before release.

Alpha testing	Beta testing
Conducted at the developer's site.	Conducted at the user's site.
Performed by end users under the developer's observation.	Performed by end users without direct developer observation.
Controlled environment.	Real-world, uncontrolled environment.
Aims to find and fix errors before beta testing begins.	Aims to catch remaining issues before final release.

6.4 Black-Box and White-Box Testing

Both are complementary philosophies for *designing test cases*; neither alone is sufficient. **White-box testing** (glass-box/structural testing) designs tests from knowledge of the code's internal structure and logic, exercising specific paths, conditions and loops, so that internal operations match specification and all code paths are exercised. **Black-box testing** (behavioural/functional testing) designs tests from the functional requirements alone, with no regard for internal code, treating the software purely as inputs mapped to expected outputs; it is good at finding missing/incorrect functions, interface errors, data-structure/database-access errors, behaviour or performance errors, and initialisation/termination errors. Each finds classes of error the other is likely to miss, so both are necessary: white-box confirms internal correctness and coverage; black-box confirms the software does what the user actually needs.

White-box: basis path testing

McCabe's **basis path testing** uses **cyclomatic complexity** $V(G)$ to find the minimum number of independent paths through a module – and hence the minimum number of test cases needed so that every statement is executed at least once. White-box testing must also guarantee: every independent path is exercised at least once; every decision is exercised on both its true and false sides; every loop is exercised at its boundaries and within normal bounds; every internal data structure is exercised for validity. (Related control-structure techniques: **condition testing** exercises every logical condition; **data-flow testing** chooses paths by where variables are defined and used; **loop testing** checks simple loops by skipping them, taking 1 pass, 2 passes, a typical m passes, and $n - 1, n, n + 1$ passes at the maximum n , and checks nested loops from the innermost outward.)

Black-box: equivalence partitioning

Equivalence partitioning divides the input domain into classes expected to be handled identically by the software, then picks one representative test value per class – covering the input space with far fewer cases than testing every value. Rules: a **range** input gives one valid class plus two invalid classes

(below, above); a **specific value** gives one valid and two invalid classes; a **member-of-a-set** input gives one valid (in the set) and one invalid (not in the set) class; a **Boolean** input gives one valid (true) and one invalid (false) class.

Example: an input field accepts ages 18–60.

Class	Definition	Test value
Valid	$18 \leq \text{age} \leq 60$	30
Invalid ₁	$\text{age} < 18$	10
Invalid ₂	$\text{age} > 60$	75

Three test cases – 30, 10, 75 – now cover the entire input domain: one representative from each of the valid class and the two invalid classes.

Black-box: boundary value analysis (BVA)

Errors cluster at the *edges* of a range far more often than in its middle, so **boundary value analysis** tests exactly at and around each boundary. For a range bounded by a (lower) and b (upper), test: $a - 1$, a , $a + 1$, $b - 1$, b , $b + 1$.

Example: the same age field, valid range 18–60, so $a = 18$, $b = 60$.

Formula	$a - 1$	a	$a + 1$	$b - 1$	b	$b + 1$
Substituted ($a=18, b=60$)	$18 - 1$	18	$18 + 1$	$60 - 1$	60	$60 + 1$
Test value	17	18	19	59	60	61

Six test values – 17, 18, 19, 59, 60, 61 – exercise both boundaries and their immediate neighbours, where off-by-one errors ($>$ vs. $\geq$, $<$ vs. $\leq$) actually occur.

Coverage criteria

Statement coverage: every statement executed at least once. **Branch (decision) coverage:** every decision outcome (true *and* false) taken at least once – stronger than statement coverage. **Path coverage:** every independent path through the module executed at least once – the strongest of the three, and exactly what cyclomatic complexity counts.

Cyclomatic complexity – worked example

Cyclomatic complexity, $V(G)$

For a program's flow graph G , cyclomatic complexity can be computed three equivalent ways:

1. $V(G)$ = number of regions in the flow graph.
2. $V(G) = E - N + 2$, where E = number of edges, N = number of nodes.
3. $V(G) = P + 1$, where P = number of predicate (decision) nodes.

$V(G)$ is the number of independent paths through the module, and hence the minimum number of test cases needed for full path coverage.

Example: model a simple structure: `while` a loop-condition holds, run one `if/else`, then loop back; when the loop condition fails, exit.

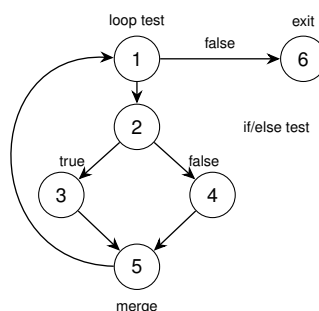


Figure 42: Constructed flow graph: node 1 = loop condition, node 2 = if/else condition, nodes 3, 4 = the two branch bodies, node 5 = merge point (loops back to node 1), node 6 = loop exit.

Step 1 – count from the drawn graph. Nodes $N = \{1, 2, 3, 4, 5, 6\}$, so $N = 6$. Edges: $1 \rightarrow 2$, $1 \rightarrow 6$, $2 \rightarrow 3$, $2 \rightarrow 4$, $3 \rightarrow 5$, $4 \rightarrow 5$, $5 \rightarrow 1$ – so $E = 7$.

Step 2 – apply $V(G) = E - N + 2$. $V(G) = 7 - 6 + 2 = 3$.

Step 3 – cross-check with $V(G) = P + 1$. The predicate (decision) nodes are node 1 (loop test) and node 2 (if/else test), so $P = 2$, giving $V(G) = 2 + 1 = 3$. ✓ matches Step 2.

Step 4 – cross-check with the regions formula. By Euler’s formula for a connected planar graph, the number of regions (faces) equals $E - N + 2$ as well: the graph encloses 2 bounded regions (the two-lane path $2-3-5$ and $2-4-5$, plus the loop-back $5-1$) and 1 unbounded outer region, giving $2 + 1 = 3$ regions total. ✓ matches Steps 2–3.

Step 5 – interpret. $V(G) = 3$ means **3 independent paths** must be tested for full path coverage:

Path	Route
1	$1 \rightarrow 6$ (loop body never runs – condition false immediately)
2	$1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 1 \rightarrow 6$ (loop runs once, if-branch taken)
3	$1 \rightarrow 2 \rightarrow 4 \rightarrow 5 \rightarrow 1 \rightarrow 6$ (loop runs once, else-branch taken)

Testing these 3 paths exercises every node and every edge of the graph at least once, satisfying full path coverage with the minimum possible number of test cases.

6.5 Agile Testing Practices

Agile methods weave testing through the whole development lifecycle instead of leaving it as one late, separate phase: testing is continuous, collaborative, and tied directly to user stories and acceptance criteria.

Test-Driven Development

Test-Driven Development (TDD)

TDD is a practice in which automated unit tests are written *before* the production code, following a short, repeated **Red-Green-Refactor** cycle.

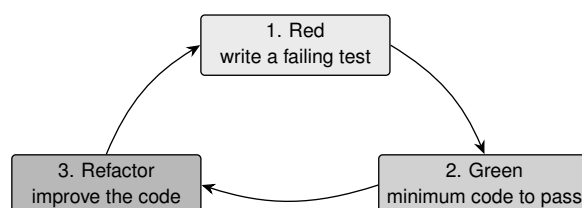


Figure 43: The TDD cycle: write a failing test (red), write the simplest code that passes it (green), then improve the code while all tests still pass (refactor) – then repeat for the next piece of functionality.

1. Red: before any production code, write a small, focused test for the next bit of desired functionality;

it must fail, since the code does not exist yet – confirming the test is valid and the requirement really is missing. **2. Green:** write the simplest possible code that makes the test pass – correctness, not elegance, is the goal. **3. Refactor:** with the passing test as a safety net, remove duplication, improve names, simplify logic; the test must still pass afterwards. **Benefits:** forces thinking about requirements/interfaces before implementation; produces a full regression-test suite as a byproduct; encourages small, loosely coupled code; gives immediate feedback when new code breaks something; cuts debugging time (the failing test pinpoints the moment of breakage); the test suite becomes living documentation of expected behaviour.

Continuous Testing and Automated Unit Testing

Continuous testing runs automated tests at every stage of the delivery pipeline, as a core part of CI/CD: a developer commits code → the CI server (e.g. Jenkins, GitHub Actions, GitLab CI) builds the project and runs the test suite → any failure halts the pipeline and notifies the developer immediately → only code that passes every test is promoted onward (integration, staging, deployment). It runs on every commit (not just at sprint end), gives feedback within minutes, spans unit/integration/sometimes acceptance tests, and enables **shift-left** testing – pushing testing earlier to cut the cost of fixing defects.

Automated unit testing uses a testing framework (**JUnit** for Java, **pytest** for Python, **NUnit** for .NET, **Jest** for JavaScript) to write, run and report tests without manual effort; tests live alongside production code and follow the **Arrange-Act-Assert** structure: **Arrange** the test data/objects/preconditions, **Act** on the unit under test, **Assert** the actual output matches the expected one. Good automated tests are: **independent** (no dependency on order or other tests' state), **fast**, **deterministic** (same result every run), **clearly named** (name states what and what's expected), and cover both normal and edge/error cases.

Refactoring for Code Quality

Refactoring

Refactoring is the disciplined restructuring of existing code's internal structure *without* changing its external behaviour – to improve readability, cut complexity and duplication, ease future extension, and pay down technical debt.

Code smells (signs refactoring is needed): **long method** (does too much – split it); **duplicate code** (same logic in multiple places); **large class** (too many responsibilities – split it); **dead code** (unused and should be removed); **feature envy** (a method uses another class's data more than its own); **magic numbers** (hard-coded values that should be named constants).

Common techniques: **extract method** (pull a fragment into a well-named method); **rename variable/method** (replace unclear names); **replace magic number with named constant**; **inline method** (fold a trivial method back into its caller); **move method** (relocate it to the class it logically belongs to); **decompose conditional** (break a complex condition into well-named methods).

```
// Before -- magic number
if (age > 18) { ... }

// After -- named constant
if (age > MIN_VOTING_AGE) { ... }
```

Refactoring must always be backed by a comprehensive automated test suite: run all tests before refactoring to confirm they pass, then re-run them after *every* small step to confirm behaviour is unchanged. This is why TDD and refactoring are inseparable in practice – TDD is what builds the test suite that makes refactoring safe.

Chapter 7 Software Quality, Assurance and Maintenance

7.1 Quality Concepts

Software quality

Software quality is an effective software process applied in a manner that creates a useful product providing measurable value for both those who produce it and those who use it. An effective process gives the infrastructure (management checks and balances, umbrella activities such as reviews and change management) for building quality in; a useful product satisfies both stated and implicit user needs reliably; and quality adds value to the producer (less maintenance, fewer bug fixes) and the user (a capability that speeds up business processes).

Garvin's five views of quality – quality is multifaceted and can be seen from any of five viewpoints:

1. **Transcendental view** – quality is immediately recognizable but cannot be explicitly defined.
2. **User view** – quality is judged by whether the product meets the end user's specific goals.
3. **Manufacturer's view** – quality means conformance to the original specification.
4. **Product view** – quality is tied to inherent characteristics (functions, features) of the product.
5. **Value-based view** – quality is measured by how much a customer is willing to pay for it.

Quality of design vs. quality of conformance

Quality of design is the degree to which the design meets the functions and features specified in the requirements model. **Quality of conformance** is the degree to which implementation follows the design, so that the resulting system meets its requirements and performance goals. Robert Glass's formula: *user satisfaction = compliant product + good quality + delivery within budget and schedule*.

Cost of quality – all costs incurred pursuing quality, plus the downstream cost of lacking it. **Prevention costs** – quality planning/coordination, complete requirements and design work, test planning, training. **Appraisal costs** – technical reviews, metrics collection and evaluation, testing and debugging. **Failure costs** – **internal** (rework, side-effect repair, failure-mode analysis – found before shipment) and **external** (complaint resolution, product return/replacement, help-line support, warranty work, loss of reputation – found after shipment).

The later a defect is found, the more it costs to fix. Boehm and Basili's data shows the relative cost rising sharply from requirements through maintenance:

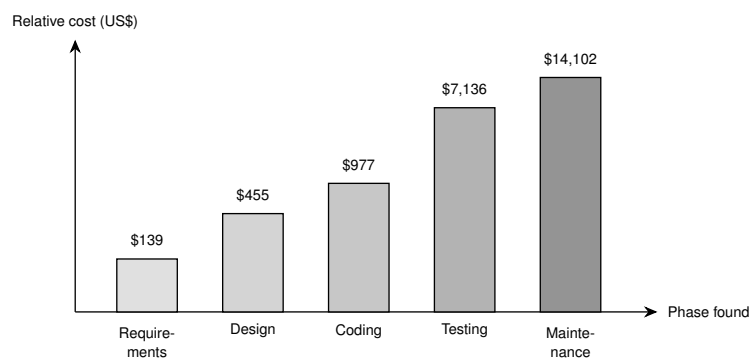


Figure 44: Relative cost of correcting a defect by the phase in which it is found (Boehm & Basili, 2001).

7.2 Quality Attributes

McCall's quality factors (McCall & Walters, 1977) organize 11 quality factors into three perspectives of a software product:

Product Operation	Product Revision	Product Transition
Correctness	Maintainability	Portability
Reliability	Flexibility	Reusability
Efficiency	Testability	Interoperability
Integrity		
Usability		

Product Operation (how well it runs): correctness – satisfies spec/mission objectives; reliability – performs without failure; efficiency – resources/code needed; integrity – controls unauthorized access; usability – effort to learn/operate. *Product Revision* (how easy to change): maintainability – effort to locate/fix an error; flexibility – effort to modify; testability – effort to test. *Product Transition* (adapting to new environments): portability – effort to move across platforms; reusability – reusable in other applications; interoperability – effort to couple with another system.

ISO 25010 defines two quality models. **Quality in Use** (5 characteristics, product in context of use): Effectiveness, Efficiency, Satisfaction, Freedom from risk, Context coverage. **Product Quality** (8 characteristics, static/dynamic nature of the product): Functional suitability, Performance efficiency, Compatibility, Usability, Reliability, Security, Maintainability, Portability.

7.3 Reviews, Inspections and QA Concepts

Software Quality Assurance (SQA)

SQA is a planned and systematic pattern of actions required to ensure high quality in software – an umbrella activity applied throughout the software process. It encompasses: an SQA process; QA/QC tasks (technical reviews, multitiered testing); effective engineering practice (methods and tools); control of work products and their changes; a procedure ensuring compliance with standards; and measurement/reporting mechanisms.

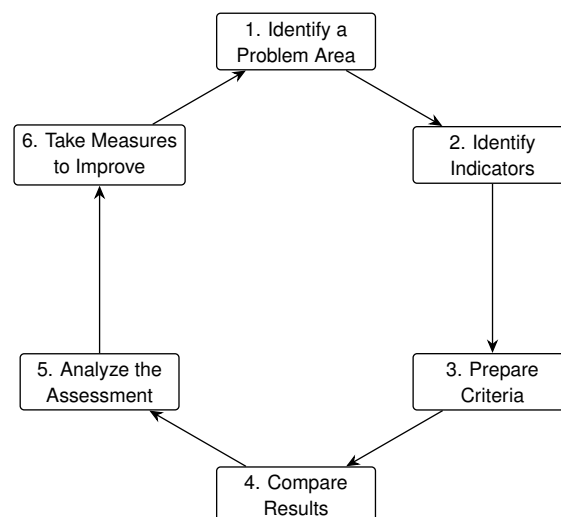


Figure 45: The six-stage SQA process: a continuous loop of measurement and improvement.

Ten elements of SQA:

Standards – conformance to IEEE/ISO etc.	Education – process-improvement training
Reviews and audits – peer reviews + SQA audits	Vendor management – quality from external vendors
Testing – properly planned and conducted	Security management – process/technology for security
Error/defect collection and analysis	Safety – assess/reduce impact of failure
Change management	Risk management – risk activities and contingency plans

Six SQA-group tasks: (1) prepares the project's SQA plan (evaluations, audits, reviews, standards, error reporting/tracking, feedback); (2) participates in developing the process description and reviews it for compliance; (3) reviews engineering activities against the defined process; (4) audits designated work products for compliance; (5) ensures deviations are documented and handled per procedure; (6) records noncompliance and reports it to senior management.

Formal Technical Review (FTR) – a structured SQA activity conducted by a small group of peers, to uncover errors in function/logic/implementation, verify the software meets requirements, ensure representation to predefined standards, and make projects more manageable. Conducted in five steps: **Planning** (team of 3–5 incl. moderator, producer, reviewers; materials distributed beforehand) → **Preparation** (each reviewer studies the product alone, ≤ 2 hours) → **Review meeting** (moderator runs a strict agenda, recorder logs defects, ≤ 2 hours, focused on finding problems not solving them) → **Decision** (accept; rework – moderator verifies, no re-review; or reject – rework and re-review) → **Follow-up** (producer fixes defects, moderator verifies, a review summary report is produced). *Guidelines:* review the product not the producer; keep an agenda; limit debate; note problems without solving them on the spot; take written notes; limit participants and insist on preparation; keep a checklist per review type.

Statistical SQA collects and categorizes defect data, traces each defect to its cause (e.g. nonconformance to spec, design error, standards violation, poor communication), and applies the **Pareto principle** – roughly 80% of defects trace to 20% of possible causes, “**the vital few**” – to focus corrective action where it yields the greatest improvement for the least effort; as the vital few are fixed, new candidates rise to the top, driving continuous improvement.

Six Sigma is the most widely used statistical-quality strategy (the name comes from six standard deviations, i.e. 3.4 defects per million). **DMAIC** improves an *existing* process; **DMADV** (Define-Measure-Analyze-Design-Verify) is used to design a *new* one.

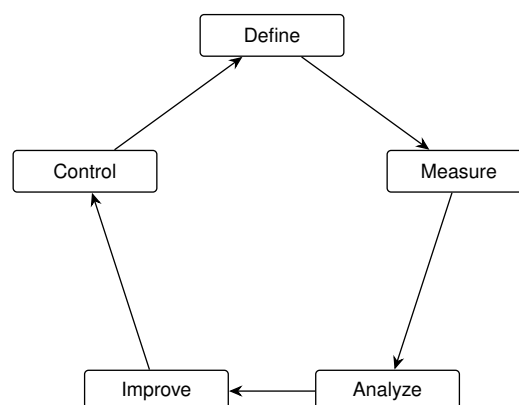


Figure 46: The DMAIC cycle: Define the problem → Measure it → Analyze the cause → Improve (fix) → Control (sustain), then repeat.

7.4 ISO Standards and CMMI Levels

ISO 9001:2015 is a generic quality-management-system standard, applicable to any organization, requiring a documented quality system, defined objectives and responsibilities, review mechanisms, adequate resources, process controls and a continual-improvement mechanism – built around the **Plan-Do-Check-Act (PDCA)** cycle. A company is ISO-certified after third-party audit and stays certified through semiannual surveillance audits.

CMMI – Capability Maturity Model Integration

CMMI (Software Engineering Institute) is a process-improvement framework defining five staged **maturity levels**, each a foundation for the next.

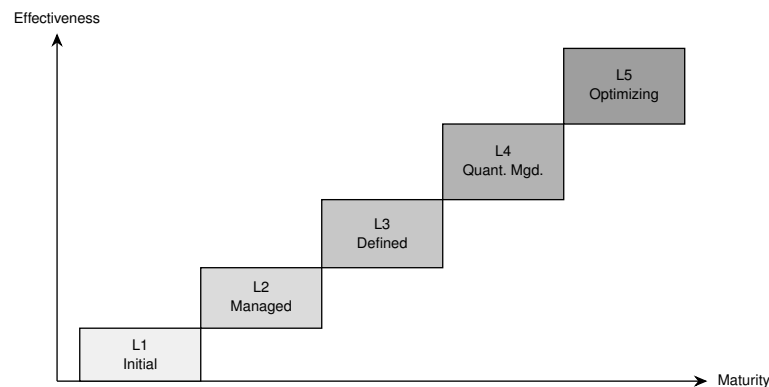


Figure 47: CMMI’s five maturity levels form an ascending staircase of process capability.

L1 Initial – ad hoc, chaotic; success depends on individual heroics. **L2 Managed** – basic project management (cost, schedule, scope) tracked; earlier successes repeatable. **L3 Defined** – management and engineering processes documented, standardized, integrated into the organization’s standard process. **L4 Quantitatively Managed** – quantitative objectives set for quality/performance; measures collected and statistically analyzed. **L5 Optimizing** – continuous improvement from quantitative feedback and piloted innovations; weaknesses fixed proactively.

ISO 9001:2015	CMMI
Quality management system standard.	Process-improvement framework.
Goal: quality management and customer satisfaction.	Goal: process capability and maturity improvement.
Applicable to any industry, any size.	Mainly IT, software, engineering, defense.
Certification: pass/fail third-party audit.	Appraisal: maturity rating on a 1–5 scale.

The two are complementary, not competing: ISO 9001 is an overarching quality framework, while CMMI supplies granular practices to mature the software process.

7.5 Software Maintenance and Its Types

Software maintenance

Software maintenance is the modification of a software product after delivery, to correct faults, improve performance or other attributes, or adapt the product to a changed environment. It is a continuous phase beginning at delivery and typically consumes 60–80% of total lifecycle cost – the most expensive phase of the lifecycle.

Type	Definition and example
Corrective	Reactive – diagnoses and fixes bugs (design, logic or code errors) found after delivery, in response to problem reports or monitoring. <i>Example:</i> Fixing a payroll calculation error that causes wrong tax deductions.
Adaptive	Modifies the software to keep it usable in a changed environment – new OS, hardware, regulations, DBMS, or cloud migration. <i>Example:</i> Updating a desktop app for a new OS version, or to comply with a new data-privacy regulation.
Perfective	Improves performance, maintainability or other attributes, or adds features requested by users; driven by evolving business needs. <i>Example:</i> Adding a reporting module to an ERP system; optimizing a search algorithm for speed.
Preventive	Proactive – restructures code, optimizes, and updates documentation before problems occur, to reduce technical debt. <i>Example:</i> Refactoring legacy code; updating libraries before they go unsupported; adding missing error handling.

7.6 Maintenance Effort and Lifecycle Considerations

Maintenance type	Share of effort
Perfective (enhancements, new features)	~50%
Corrective (bug fixes)	~21%
Adaptive (environment changes)	~25%
Preventive (future-proofing)	~4%

Factors affecting maintenance effort: (1) **Application domain** – complex domains need more effort to understand/modify; (2) **Staff stability** – turnover means new staff must re-learn the codebase (can consume up to 50% of effort); (3) **Software age and structure** – older, poorly structured programs cost more, per Lehman’s **Law of Increasing Complexity**; (4) **Documentation quality** – poor documentation makes safe change difficult; (5) **Programming language/tools** – obsolete ones mean fewer skilled maintainers; (6) **Reliability of the original software** – high initial defect density needs more corrective work; (7) **Hardware/software environment changes** – rapid platform change increases adaptive maintenance.

Lehman’s Laws of Software Evolution (two relevant to maintenance): the **Law of Continuing Change** – a system must be continually adapted or it becomes progressively less useful; and the **Law of Increasing Complexity** – as a system evolves its complexity increases unless active work is done to maintain or reduce it. Together they explain why maintenance is unavoidable and why preventive maintenance (restructuring, refactoring) is needed to keep later corrective/adaptive/perfective work affordable. Other lifecycle considerations: maintenance must be *planned from delivery*, not treated as an afterthought; disciplined **configuration management** (version control, change management, release planning) is essential; when maintaining a legacy system costs more than replacing it, **re-engineering** (restructuring/rewriting without changing functionality) or retirement should be considered; every change needs an **impact analysis** beforehand and **regression testing** afterward to confirm existing functionality still works.

Software reliability – the probability of failure-free operation of a program in a specified environment for a specified time; unlike hardware, it stems from design/implementation defects, not physical wear. **MTBF** (Mean Time Between Failure) = $MTTF + MTTR$ (mean time to failure +

mean time to repair). **Availability** = $\frac{\text{MTTF}}{\text{MTTF} + \text{MTTR}} \times 100\%$ – more sensitive to MTTR, so it is an indirect measure of maintainability. **FIT** (Failures In Time) – number of failures expected per one billion (10^9) device-hours of operation.

Chapter 8 Software Configuration Management

8.1 Software Configuration Management

Software Configuration Management (SCM)

Software Configuration Management is an umbrella software-engineering activity that identifies, organises and controls change to the work products produced throughout the software process, so that the integrity and traceability of the software are maintained as it evolves. Every item placed under SCM control – source code, requirements/design documents, test plans and cases, build scripts, data schemas, user manuals – is called a **Software Configuration Item (SCI)**. A collection of SCIs that has been reviewed and agreed upon, and that thereafter serves as a fixed reference point for further development, is called a **baseline**; a change to a baselined SCI can be made only through a formal, controlled change procedure.

SCM matters because software changes constantly – new requirements, defect fixes, environment upgrades – and uncontrolled change causes confusion (which version is “correct?”), lost work (two engineers overwrite each other’s changes) and broken builds. SCM answers four recurring questions: *What* changed? *Who* changed it? *When* was it changed? *Why* was it changed?

The SCM framework – five activities.

1. **Identification** – name and uniquely label each SCI (source files, documents, tools) and organise them into a structure so any item and version can be located.
2. **Version control** – combine procedures and tools to manage the different versions of SCIs that are created as the project proceeds.
3. **Change control** – a formal, gated procedure (request, evaluation, approval, implementation) through which change to a baselined item is proposed and made.
4. **Configuration auditing** – independently checks that an SCI conforms to its specification and that changes have been correctly recorded and applied, before it is added to a baseline.
5. **Status accounting/reporting** – maintains a record of what happened to each SCI (when it changed, why, by whom) and reports it, giving visibility to managers and engineers.

8.2 Version Control and Continuous Integration

Version control

Version control (revision control) is the practice of recording changes to files over time in a **repository**, so any earlier version can be recovered, changes can be compared, and multiple engineers can work on the same codebase concurrently without overwriting each other’s work.

Two models exist. In a **centralized** system (*Example*: Subversion/SVN, CVS) a single central repository holds the authoritative history; developers check out a working copy, make changes and commit directly back to that one server – simple to reason about, but the server is a single point of failure and offline work is limited. In a **distributed** system (*Example*: Git, Mercurial) every developer clones a **full** copy of the repository, including its entire history; commits are made locally and later **pushed** to, or **pulled** from, any remote copy (typically a shared “origin”). This gives fast local operations, full offline capability, and resilience – there is no single point of failure – which is why Git is the de-facto standard in industry today.

Continuous Integration (CI)

Continuous Integration is the practice of merging all developers’ working copies into a shared mainline (e.g. Git’s main/develop branch) frequently – ideally several times a day – with each merge

automatically built and tested by a CI server. It builds directly on version control: without a shared repository and cheap branching/merging, frequent integration would not be practical.

CI catches integration errors early, when they are small and cheap to fix, instead of during a late, painful “integration phase”. A CI loop repeats continuously as engineers work:

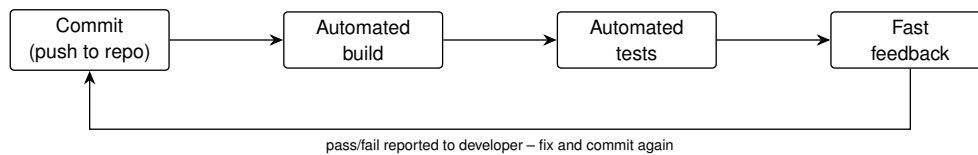


Figure 48: The continuous integration loop: every commit is automatically built, tested and reported on.

If the build or tests fail, the team is alerted immediately and fixing the mainline takes priority over new work – this keeps the shared codebase always in a known, working state.

8.3 Change Management Process

Change management

Change management is the formal process by which a proposed change to a baselined configuration item is requested, evaluated, approved (or rejected), implemented and verified, so that change happens in a controlled, traceable way rather than ad hoc.

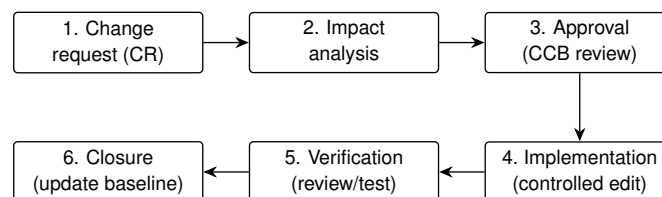


Figure 49: The change management process: from change request to a re-baselined, verified item.

A **Change Control Board (CCB)** – typically including a project lead, developers, testers and, for major changes, the customer – weighs the impact analysis (cost, schedule, risk, affected SCIs) against the benefit before authorising work; only approved changes may be implemented against a baseline. *Example:* A customer submits a CR asking for an extra payment method; impact analysis flags three modules and two days of work; the CCB approves it; a developer implements and tests the change; on verification it is merged and the baseline (and its documentation) is updated, closing the CR.

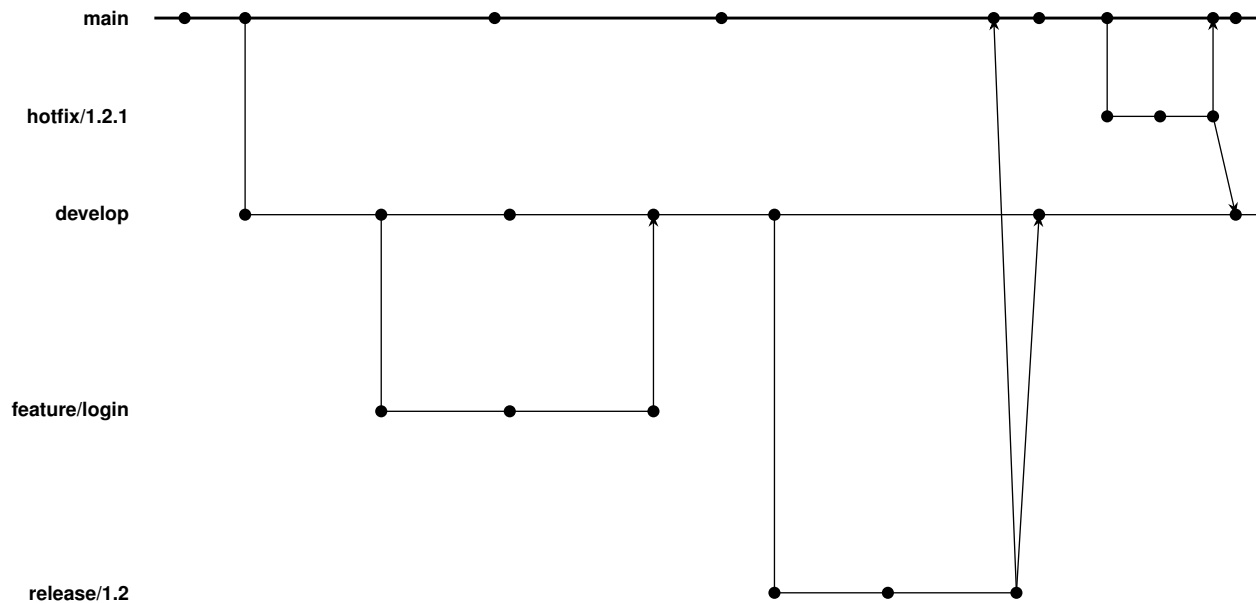
8.4 Branching Strategies (Git-Flow Basics)

Branching strategy

A **branching strategy** is a team convention for how and when branches are created, named and merged in a version-control repository, so that parallel work (new features, releases, urgent fixes) can proceed without destabilising the code that is already working. **Git-flow** is a widely used strategy built around two permanent branches and three types of supporting, temporary branches.

- **main/master** – always reflects the latest *released*, production-ready code; every commit on it is (in principle) deployable.
- **develop** – the integration branch; holds the latest delivered development changes for the next release.
- **feature branches** – branch off **develop** for a single feature/story; merge back into **develop** when complete; never interact with **main** directly.

- **release branches** – branch off **develop** when it has enough features for a release, to allow final testing/bug-fixing/version bumping; merge into *both* **main** (tagged as a release) and back into **develop**.
- **hotfix branches** – branch off **main** to patch an urgent production bug without waiting for the next release cycle; merge into *both* **main** (tagged) and back into **develop**, so the fix is not lost in future releases.



• Solid horizontal line = commit history on a branch • Diagonal (plain) = branch created • Diagonal (arrow) = merge

Figure 50: Git-flow: **main** and **develop** are permanent; **feature**, **release** and **hotfix** branches are temporary and merge back in as shown.

Git-flow gives large or release-scheduled teams a predictable, disciplined workflow, at the cost of some process overhead; smaller teams practising continuous deployment often prefer lighter schemes such as **trunk-based development** or **GitHub flow**, where short-lived feature branches merge directly into a single mainline that is kept always releasable.

Chapter 9 Recent Trends

9.1 DevOps and Continuous Deployment Pipelines

DevOps

DevOps is a set of cultural philosophies, practices and tools that combines software **Development** and IT **Operations**, aiming to shorten the systems-development life cycle and deliver features, fixes and updates frequently, in close alignment with business goals. It breaks down the traditional wall between developers (who want to ship change) and operations staff (who want stable systems), replacing hand-offs with shared responsibility, automation, and continuous feedback from production back into planning.

DevOps is usually pictured as an unending loop – there is no “final” release; the system is continuously planned, built, released and observed, and what is learned in operation feeds the next planning cycle.

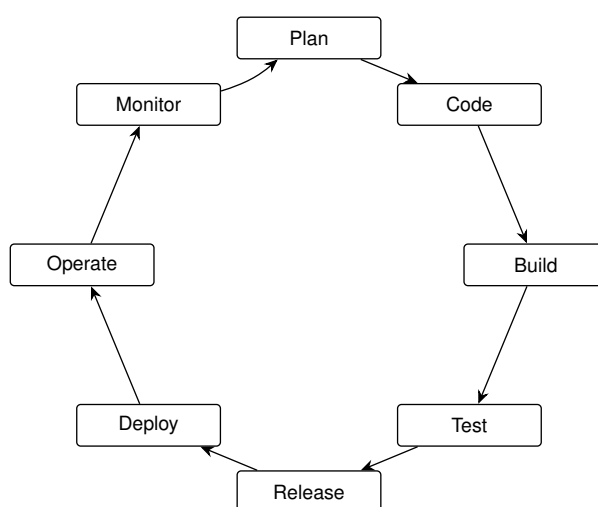


Figure 51: The DevOps lifecycle: a continuous loop from planning through to monitoring, feeding back into the next plan.

Continuous deployment extends this loop so that every change which passes the automated pipeline is released to production automatically, with no manual gate – the fastest form of delivery, relying heavily on strong automated testing and monitoring to catch problems immediately after release.

9.2 Continuous Integration/Continuous Delivery (CI/CD) in Modern Projects

CI vs. CD vs. continuous deployment.

- **Continuous Integration (CI)** – developers merge code into a shared mainline frequently; each merge is automatically built and tested (recall §8.2).
- **Continuous Delivery** – extends CI so that, after passing automated tests, the build is automatically packaged into a release-ready artefact and can be deployed to production *at the push of a button* – a human still decides *when* to release.
- **Continuous Deployment** – goes one step further: every change that passes the pipeline is deployed to production *automatically*, with no manual approval step.

A **CI/CD pipeline** is the automated sequence of stages a change passes through from commit to production; modern projects run it on tools such as Jenkins, GitLab CI, GitHub Actions or Azure DevOps, and it forms the automation backbone of DevOps.

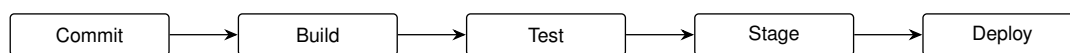


Figure 52: A typical CI/CD pipeline: each stage must pass before the change advances to the next.

A failure at any stage stops the pipeline and reports back to the developer immediately, keeping feedback fast and production protected from broken changes.

9.3 Cloud-Native, Microservices and Infrastructure as Code

Cloud-native

Cloud-native systems are designed from the outset to run on cloud infrastructure, exploiting its elasticity. Applications are packaged as **containers** (lightweight, portable units bundling code with all its dependencies, *Example:* Docker) and managed at scale by an **orchestrator** (*Example:* Kubernetes), which handles deployment, scaling, load balancing, self-healing (restarting failed containers) and rolling updates automatically.

Microservices architecture

A **microservices architecture** structures an application as a collection of small, independently deployable services, each built around a single business capability, owning its own data, and communicating with the others over lightweight APIs (typically HTTP/REST or messaging). This contrasts with a **monolithic** architecture, where all functionality is built and deployed as one single unit. **Adv:** independent development, deployment and scaling of each service; a failure in one service need not bring the whole system down; teams can use different technologies per service. **Disadv:** added operational complexity (many services to deploy, monitor, version); network latency and failure between services; harder to maintain data consistency across services.

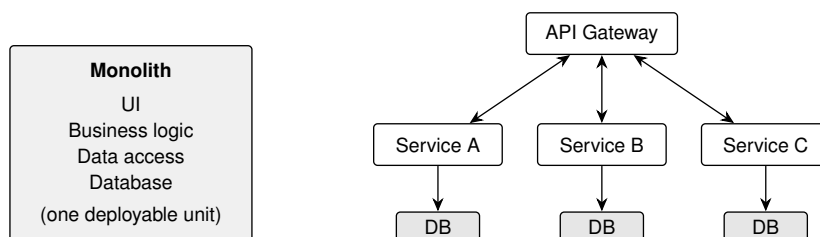


Figure 53: Monolithic architecture (single unit) versus microservices (small, independently deployable services, each with its own data, behind a gateway).

Infrastructure as Code (IaC)

Infrastructure as Code manages and provisions servers, networks and other infrastructure through machine-readable configuration files, kept under version control, rather than by manually configuring hardware or clicking through a console. Running the same configuration always produces the same environment – eliminating configuration drift and making infrastructure changes reviewable, repeatable and auditable, just like application code. *Example:* Terraform (provisioning cloud resources), Ansible (configuring servers).

9.4 AI-Assisted Software Development

Modern development tools increasingly embed AI assistance directly into the workflow: **code completion/generation** suggests or writes code from a natural-language prompt or partial code; **AI pair programming** assistants (*Example:* GitHub Copilot) offer real-time suggestions and can explain or refactor existing code; and **automated review/test generation** tools can flag likely defects, suggest fixes, or generate unit test cases from source code. These tools speed up routine coding and lower the barrier to writing tests, but AI-generated code and suggestions can be subtly wrong, insecure or based

on outdated patterns – so **human review** remains essential: a developer must still understand, verify and take responsibility for anything an AI tool produces before it is merged.

9.5 Low-Code/No-Code and Green Software Engineering

Low-code/no-code platforms

Low-code platforms let developers build applications mostly through visual, drag-and-drop tools and configuration, writing only small amounts of custom code for special cases; **no-code** platforms remove hand-written code entirely, targeting non-programmers (“citizen developers”). They speed up delivery of simple, common applications (forms, workflows, internal tools) but offer less flexibility and control than traditional coding for complex or highly customised systems.

Green software engineering

Green software engineering designs and operates software to minimise its energy consumption and carbon footprint. Practices include writing **efficient algorithms and code** (lower CPU/memory use for the same task), **right-sizing** cloud resources to actual demand instead of over-provisioning, and **carbon-aware computing** – scheduling non-urgent workloads for times or locations where the electricity supply is cleaner. It treats energy efficiency as a software quality attribute alongside performance, reliability and maintainability.

Knowledge Check: 40 Multiple Choice Questions

Choose one option per question. Answers are at the end. Questions 1–3 cover Chapter 1, 4–10 Chapter 2, 11–15 Chapter 3, 16–18 Chapter 4, 19–26 Chapter 5, 27–31 Chapter 6, 32–34 Chapter 7, 35–37 Chapter 8, and 38–40 Chapter 9.

Chapter 1: Introduction

1. The gap between the rapidly rising demand for software and the industry's limited capacity to deliver it reliably, on time and within budget is commonly called the:
(A) software process (B) software crisis
(C) software myth (D) software maintenance backlog
2. Software that must be changed and kept operational because it is central to a business, even though its original documentation may be poor, is called:
(A) legacy software (B) embedded software
(C) system software (D) shrink-wrapped software
3. "Once we write the program and get it to work, our job is done" is an example of a:
(A) customer myth (B) practitioner myth
(C) management myth (D) process myth

Chapter 2: The Software Process

4. The software process model in which each phase (requirements, design, coding, testing, maintenance) must be completed before the next begins, with no planned overlap, is the:
(A) spiral model (B) incremental model
(C) waterfall model (D) prototyping model
5. A process model that delivers the software in a series of usable, progressively more complete builds, each adding functionality to the previous one, is the:
(A) incremental model (B) waterfall model
(C) V-model (D) Big Bang model
6. In the spiral model, each loop (cycle) around the spiral explicitly includes a dedicated activity for:
(A) final delivery only (B) code freeze
(C) legacy migration (D) risk analysis
7. The V-model pairs each development (left-hand) phase with a corresponding testing (right-hand) phase; the phase paired with system design is:
(A) system/integration testing (B) unit testing
(C) acceptance testing (D) requirements review
8. According to the Agile Manifesto, which of the following is valued more than the item paired with it?
(A) comprehensive documentation over working software (B) following a plan over responding to change
(C) contract negotiation over customer collaboration (D) individuals and interactions over processes and tools
9. In Scrum, the time-boxed period (typically 2–4 weeks) during which a potentially shippable increment is produced is called a:
(A) backlog (B) sprint
(C) release (D) stand-up

10. The Extreme Programming (XP) practice in which two developers work together at one workstation, one writing code while the other reviews it in real time, is called:

- (A) continuous integration
- (B) collective ownership
- (C) test-driven development
- (D) pair programming

Chapter 3: Software Requirements Engineering

11. A requirement that states what the system must do – a specific function, feature or service – is a:

- (A) non-functional requirement
- (B) domain requirement
- (C) design constraint
- (D) functional requirement

12. A requirement specifying a quality attribute such as performance, reliability, usability or security, rather than a specific function, is a:

- (A) functional requirement
- (B) user requirement (informal only)
- (C) non-functional requirement
- (D) business rule only

13. The formal document that describes the complete external behaviour of a system to be built, serving as a contract between customer and developer, is the:

- (A) Software Design Document (SDD)
- (B) Software Requirements Specification (SRS)
- (C) Software Test Plan (STP)
- (D) Software Configuration Plan (SCP)

14. A use case's main path through the system, describing the normal, expected interaction that achieves the actor's goal without errors or deviations, is called the:

- (A) alternative flow
- (B) exception flow
- (C) basic (main success) flow
- (D) extension point

15. In agile requirements engineering, a short, informal description of a feature from an end-user's perspective, typically in the form "As a [role], I want [goal], so that [benefit]", is called a:

- (A) use case
- (B) epic
- (C) user story
- (D) story map

Chapter 4: Architectural Design

16. In the repository (data-centered) architectural style, independent components communicate with each other:

- (A) directly, via point-to-point messages
- (B) only through a message queue broker
- (C) only through a shared central data store
- (D) through a shared global clock

17. Remote Procedure Call (RPC) is best described as the call-and-return (main program/subprogram) style with the key difference that:

- (A) there is no longer any control hierarchy
- (B) the calling and called components are distributed across networked machines
- (C) all calls become asynchronous message broadcasts
- (D) the subprograms share one address space

18. In the Model-View-Controller (MVC) architectural pattern, the component that interprets user input, invokes the Model and selects the View to render is the:

- (A) Model
- (B) View
- (C) Gateway
- (D) Controller

Chapter 5: System Modeling

19. In a Data Flow Diagram drawn with Yourdon–DeMarco notation, a process (a function that transforms incoming data into outgoing data) is drawn as a:

- (A) rectangle
- (B) open-ended rectangle
- (C) diamond
- (D) circle (bubble)

20. In DFD rules, data is never allowed to flow directly between two data stores, or between an external entity and a data store; it must always pass through a:
- (A) process
 - (B) data flow arrow alone
 - (C) second external entity
 - (D) decision node
21. A DFD level that shows the entire system as a single process, with only the external entities and the flows crossing the system boundary, and no internal processes or data stores, is called the:
- (A) Level-1 DFD
 - (B) context diagram (Level-0)
 - (C) Level-2 DFD
 - (D) class diagram
22. The rule that every data flow entering or leaving a parent process in a DFD must also appear, in total, in its child (decomposed) diagram is called the:
- (A) cohesion rule
 - (B) balancing rule
 - (C) information-hiding rule
 - (D) traceability matrix
23. In a UML activity diagram, a filled bar that splits one flow into several concurrent flows, or synchronises several concurrent flows back into one, represents a:
- (A) fork/join
 - (B) decision (branch)
 - (C) initial node
 - (D) swimlane
24. In a UML class diagram, the relationship drawn as a solid line with a filled (solid) diamond at the “whole” end, indicating that the part cannot exist without the whole and is destroyed with it, is:
- (A) aggregation
 - (B) generalization
 - (C) composition
 - (D) plain association
25. In a UML class diagram, the relationship drawn as a hollow triangle pointing from a subclass to its superclass represents:
- (A) generalization (inheritance)
 - (B) composition
 - (C) aggregation
 - (D) dependency
26. A lightweight, index-card-sized alternative to a full class diagram that lists, for each class, only its name, its responsibilities and its collaborators is a:
- (A) CRC (Class-Responsibility-Collaborator) card
 - (B) use-case diagram
 - (C) sequence diagram
 - (D) DFD data dictionary entry

Chapter 6: Coding and Testing

27. A formal, structured code-review technique in which a moderator leads a scripted meeting through defined roles and checklists to find defects, historically shown to be highly effective, is called a:
- (A) walkthrough
 - (B) Fagan inspection
 - (C) pair programming session
 - (D) stand-up meeting
28. A black-box testing technique that divides a module’s input data into classes from which test cases are derived, on the assumption that all values in a class are processed alike, is called:
- (A) equivalence partitioning
 - (B) boundary value analysis
 - (C) basis path testing
 - (D) cyclomatic complexity analysis
29. Boundary value analysis focuses test-case selection on:
- (A) the exact centre (typical value) of each equivalence class
 - (B) only invalid, unreachable inputs
 - (C) the edges of input/output equivalence classes, not their centre
 - (D) random sampling across the whole input domain
30. For a program’s flow graph G with E edges and N nodes, the cyclomatic complexity formula is:
- (A) $V(G) = E - N + 2$
 - (B) $V(G) = N - E + 2$
 - (C) $V(G) = E + N - 2$
 - (D) $V(G) = E \times N$

31. Cyclomatic complexity $V(G)$ can equivalently be computed from the number of predicate (decision) nodes P as:

(A) $V(G) = P + 1$

(B) $V(G) = P - 1$

(C) $V(G) = 2P$

(D) $V(G) = P^2$

Chapter 7: Software Quality, Assurance and Maintenance

32. The international standard that defines a software product quality model built around characteristics such as functional suitability, performance efficiency, compatibility, usability, reliability, security, maintainability and portability is:

(A) ISO 9001

(B) ISO/IEC 25010

(C) IEEE 830

(D) ISO/IEC 12207

33. The Capability Maturity Model Integration (CMMI) level at which an organisation's software process is characterised as ad hoc, chaotic and dependent on individual heroics rather than a defined process, is:

(A) Level 3 – Defined

(B) Level 5 – Optimizing

(C) Level 2 – Managed

(D) Level 1 – Initial

34. The type of software maintenance carried out to fix defects discovered by users after release is called:

(A) adaptive maintenance

(B) corrective maintenance

(C) perfective maintenance

(D) preventive maintenance

Chapter 8: Software Configuration Management

35. The discipline of identifying, controlling, tracking and auditing changes to the work products of a software project (code, documents, baselines) throughout its lifecycle is:

(A) software quality assurance

(B) requirements engineering

(C) risk management

(D) Software Configuration Management (SCM)

36. In a continuous integration (CI) workflow, every commit pushed to the shared repository is automatically:

(A) merged directly into the production release without any checks

(B) archived and never tested until release day

(C) built, tested and reported on

(D) reviewed only by the original author

37. In the Git-flow branching strategy, the two permanent branches are main (production-ready code) and:

(A) feature (one branch per new feature, kept forever)

(B) hotfix (kept forever as a permanent branch)

(C) release (kept forever as a permanent branch)

(D) develop (the integration branch for the next release)

Chapter 9: Recent Trends

38. The cultural and technical movement that unifies software development (Dev) and IT operations (Ops) into a continuous loop of planning, building, testing, releasing and monitoring is:

(A) Scrum

(B) the V-model

(C) Cleanroom engineering

(D) DevOps

39. An architectural style that structures an application as a collection of small, independently deployable services, each owning its own data and communicating over lightweight APIs, is called:

(A) a monolithic architecture

(B) the repository style

(C) microservices architecture

(D) the layered style

40. Managing and provisioning servers, networks and other infrastructure through version-controlled, machine-readable configuration files instead of manual setup is called:

(A) Platform as a Service (PaaS) only

(B) Infrastructure as Code (IaC)

(C) Software as a Service (SaaS) only

(D) Continuous Deployment

Answer Key

1	B	2	A	3	C	4	C	5	A
6	D	7	A	8	D	9	B	10	D
11	D	12	C	13	B	14	C	15	C
16	C	17	B	18	D	19	D	20	A
21	B	22	B	23	A	24	C	25	A
26	A	27	B	28	A	29	C	30	A
31	A	32	B	33	D	34	B	35	D
36	C	37	D	38	D	39	C	40	B